Multiclass Classification of Celestial Objects Using Gaia-like Tabular Features

University of Plymouth

Final Year Project

Author: Riccardo Mancini
Supervisor: Vassilis Cutsuridis

January 2026

Contents

1 Acknowledgments **4**

2 Abstract **4**

3 Introduction **6**

4 Background and Related Work **8**
4.1 Supervised Learning basics 9
4.2 Multiclass Classification Basics 9
4.3 Overfitting, Underfitting, and Generalisation 10
4.4 Why Macro Metrics Matter 11
4.5 Model Families and Related Work Context 12
4.6 Cross-validation and Held-Out Test Rationale 13

5 Data Cleaning and Processing **16**
5.1 Data-specific cleaning scripts 16
5.2 Exoplanet Gaia enrichment 17
5.3 Column harmonization in notebook 17
5.4 Final multiclass preprocessing 18
5.5 Missing values: imputation vs row removal 18
5.6 Scaling policy by model type 19
5.7 Label encoding for MLP 19
5.8 Leakage prevention 20
5.9 Section Summary 20

6 Methodology and Experimental Design **22**
6.1 Experimental Objective 22
6.2 Dataset Construction for Experiments 22
6.3 Train/Validation/Test Protocol 22
6.4 Models Evaluated 23
6.5 Scaling and Encoding Policy 23
6.6 Evaluation Metrics 25
6.7 Leakage Prevention and Reproducibility 25
6.8 Methodology Summary 25

7 Implementation **26**
7.1 Notebook structure 26
7.2 Reusable Evaluation Functions 27
7.3 CV/Test Evaluation Skeleton 28
7.4 Result Aggregation and Plotting 28
7.5 Saving Outputs 29
7.6 Runtime Considerations 29
7.7 Implementation Summary 30

8 Results **30**
8.1 Preliminary Binary experiments 31
8.2 Dummy baseline 31
8.3 Main Multiclass Model Results 32
8.4 Extended Model Results 36
8.5 MLP Scaler Comparison 37
8.6 Confusion Matrix and Error Analysis 40
8.7 Results Summary 42

9 Discussion **43**
9.1 Overall Performance Trends 43
9.2 Why kNN and Random Forest Performed Strongly 43
9.3 Effect of Dataset Size 44
9.4 Overfitting and Generalisation 45
9.5 Effect of scaling on MLP 46
9.6 Fairness of the Model Comparison 46
9.7 Practical Meaning of the Results 47
9.8 Research Questions Revised 48

10 Ethical, Legal, and professional Considerations **50**

11 Limitations and Future Work **50**

12 Conclusion **53**

A Additional Multiclass Results **56**
A.1 All-Model Metric Comparisons 56
A.2 MLP Scaler Comparison 60
A.3 Additional Confusion Matrices 61

B Preliminary Binary Experiment Outputs **62**
B.1 Asteroids vs Comets 62
B.2 Host Stars vs Non-host Stars 62
B.3 Quasars vs Stars 62

C Generative AI Declaration **66**

1 Acknowledgments

I want to thank my supervisor, Vassilis Cutsuridis, and my family for supporting me through this journey.

2 Abstract

This project investigates the automated classification of celestial objects using supervised machine learning on tabular astronomical data. The main focus is a multiclass classification task that involves three classes; star, exo_host, and quasar. Data were sourced from Gaia-based star and quasar catalogues and an exoplanet_-host catalogue, then cleaned and aligned to a shared feature space to ensure a fair comparison across classes. The key preprocessing steps included column compatibility, handling missing values, feature selection, and balanced sampling.

The multiclass experiment was evaluated using balanced subsets of increasing size (10,000, 20,000, 37,800 total samples), with stratified splitting and cross-validation to preserve class distributions and reduce evaluation bias. A held-out test set was used for the final performance assessment. The models compared included a Dummy Classifier baseline, Logistic Regression, Random Forest, and Multi-Layer Perceptron (MLP), with additional exploratory comparisons using K-Nearest Neighbours, Decision Tree, and Support Vector Machines.

Performance was measured using accuracy, macro precision, macro recall, and macro F1-score, with confusion matrix analysis for class-level error interpretation. On the largest balanced subset, kNN with MinMax scaling achieved the best held-out test macro F1 (0.9997), while Random Forest achieved the strongest cross-validation macro F1 (0.9993). These findings show that careful data cleaning, feature alignment, and stratified evaluation are critical to reliable astronomical classification, and that classical machine learning models remain highly effective for structured celestial datasets.

Word Count

Word count: 10,644

Code Repository

GitHub repository: `https://github.com/Ric-cpu/Celestial-Object-Identification`

3 Introduction

Modern astronomy has entered a data-intensive era (Gaia Collaboration 2016; Gaia Collaboration *et al.* 2023). Large sky surveys and space missions now produce catalogues containing millions to billions of objects, each with positional, photometric, and astrometric measurements. This scale creates a clear opportunity for machine learning, because manual inspection and hand-crafted rule systems are no longer practical for high-volume classification tasks (Gaia Collaboration 2016; Gaia Collaboration *et al.* 2023; Bishop 2006; James *et al.* 2021). In this context, reliable automated classification is not only a convenience, but also enabling downstream scientific analysis, including candidate filtering, anomaly detection, and targeted follow-up observations.

A major challenge in building such classifiers is that astronomical data is often heterogeneous. Different catalogues use different schemas, naming conventions, quality flags, and missing_values patterns. Even when two catalogues refer to physically similar measurements, they may store them under different column names or completeness levels. As a result, the central problem is not only model selection, but also data engineering quality: the classification pipeline must be built on a consistent and defensible feature specification.

This project addresses the problem through a multiclass classification study on the three celestial object categories: star, exo_host, and quasar. The multiclass setting is important because it is closer to realistic catalogue workflows than isolated binary tasks. Rather than training the model to separate only one pair of classes, the project evaluates whether a single supervised pipeline can distinguish all three categories under shared constraints. The selected labels are:

- star
- exo_host
- quasar

The project uses Gaia-like tabular features that are present in all three classes after cleaning and alignment. This shared feature approach is intentional. It ensures that model comparison is fair and not inflated by class-specific field available only in one dataset. The resulting experiments, therefore, test classification performance under a controlled and comparable input space. The scope of the work includes the

full processing from raw and cleaned catalogue files to the final model evaluation, with an emphasis on methodological rigor. The study is designed to answer not only which model performs best, but also how robust the performance is across dataset scale and preprocessing choices. To support this, multiclass experiments are run on multiple balanced subset sizes, and evaluation uses stratified splitting, cross-validation, and held-out testing.

The main objectives of the project are:

- Clean and align heterogeneous astronomical datasets into a consistent multiclass-ready schema.
- Construct fair multiclass experiments using balanced sampling between classes.
- Evaluate the behaviour of the model on multiple scales of the data set.
- Compare performance across model families, including linear, tree-based, and neural approaches.
- Assess overfitting and generalisation using stratified K-fold cross-validation and an untouched test set.
- Examine the impact of feature scaling choices, especially for MLP.

To integrate these objectives, the project compares a baseline and several supervised models. A dummy baseline is included to establish a minimum reference level and to verify that the learned models provide meaningful improvement beyond trivial strategies. The initial core supervised models are Logistic Regression, Random Forest, and Multi-Layer Perceptron (MLP). Additional extended models, including kNN + MinMax scaling, Decision Tree, and SVM variants, are included to broaden the comparative analysis. The metrics are reported using accuracy, macro precision, macro recall, and macro F1, with confusion matrix analysis for class-level error interpretation.

The project is guided by the following research questions:

- 1. Which classical machine learning model performs best on the selected shared Gaia-like feature set for multiclass classification of stars, exo_host, and quasars?

- 2. How does the model performance change as dataset size increases from small to medium to large balanced subsets?
- 3. Does feature scaling choice affect MLP performance in a meaningful way when all other experimental conditions are held constant?

These questions are relevant both practically and methodologically. Practically, they identify a strong candidate model for this specific multiclass setting. Methodologically, they test whether performance claims remain stable across data scales and processing decisions, which is essential for credible reporting in applied machine learning.

This report is organized as follows. The next section presents background concepts and related work relevant to supervised multiclass classification and overfitting control. The data section then describes source catalogues, label design, class distributions, feature availability, and key quality constraints. The preprocessing section details cleaning, schema alignment, missing data handling, and model-specific scaling strategy. The methodology section defines the experimental protocol, including the construction of balanced subsets, stratified train/validation/test workflow, and evaluation metrics. The results are then reported for the baseline and supervised models across the dataset sizes, followed by confusion matrix analysis and scaler comparison experiments. The discussion interprets performance trends, model trade-offs, and generalisation behaviour. The report ends with limitations, future work, and a final conclusion.

4 Background and Related Work

Modern astronomy operates in a high-volume data regime in which large surveys produce catalogues containing millions to billions of records. In this setting, manual inspection is not scalable, and automated classification becomes a necessary step for downstream scientific workflows. Classical and modern machine learning methods provide practical tools for this problem, especially when observations are available in structured tabular form, `star`, `exo_host`, and `quasar`, using shared Gaia-like features after schema alignment and cleaning.

This section provides the technical background needed to justify the experimental design and model choices and covers supervised learning, multiclass classification, overfitting and generalisation, metric selection (with emphasis on

macro-averaged metrics), model family rationale, and the evaluation strategy based on stratified K-fold cross-validation plus a held-out test set.

4.1 Supervised Learning basics

Supervised learning is the task of learning a function that maps the input features X to the target labels y using labeled examples Bishop 2006; James *et al.* 2021. In this project, each object is represented by tabular features such as right ascension, declination, parallax, proper motion, and Gaia G-band magnitude, and the model predicts a class label (`star`, `exo_host`, or `quasar`).

A supervised learning pipeline typically includes data preparation, model training, validation, and final testing. Data preparation is especially important when combining heterogeneous sources, because inconsistent schemas and missing values can produce model error if left untreated. For this reason, model performance should be interpreted as the result of both algorithm selection and data-engineering quality.

Another important concept is generalisation: a model should perform well not only on seen training data but also on unseen data sampled from the same distribution. A highly complex model can fit training examples very closely, but if it captures noise rather than signal, and test performance degrades. This pushes strong evaluation protocol design, not just training accuracy.

In applied settings, supervised learning is therefore a combined optimization problem: Selecting features that are informative and consistent, selecting models that are suitable for the data geometry, and using validation strategies that produce robust estimates of real-world behaviour.

4.2 Multiclass Classification Basics

Multiclass classification extends binary classification to problems with three or more mutually exclusive classes. Instead of deciding between class 0 and class 1, the model estimates which of K classes best match each sample. In this dissertation, $K = 3$; `star`, `exo_host`, and `quasar`.

Many algorithms are naturally multiclass (for example, decision tree and random forests), while others are built from binary components internally (for exam-

ple, one-vs-rest or multinomial formulations), regardless of the implementation details, prediction quality must be assessed both globally and per class. Confusion matrices are particularly useful because they show which class pairs are being confused, not only how often mistakes occur.

The multiclass setting is more realistic than isolated binary tasks for astronomical pipelines, where catalogues often contain diverse object populations. It also imposes stricter fairness requirements: a model should not appear strong overall while failing on one class. This is why class-aware metrics and stratified sampling are important components of the methodology.

A further practical issue is label definition. This project uses explicit, fixed labels (star, exo_host, quasar) and a shared feature set available across all three classes. This avoids unfair comparisons where one class benefits from extra features unavailable to others. The shared-feature design intentionally prioritizes comparability and methodological rigor over maximal richness.

4.3 Overfitting, Underfitting, and Generalisation

Overfitting occurs when a model learns the noise of the training data, and fails to generalize to unseen data examples. Underfitting occurs when a model is too simple to capture the underlying structure. The goal is an appropriate bias-variance balance Bishop 2006; James *et al.* 2021.

In practice, overfitting often appears as a gap between training and validation/test performance. A model with near-perfect training accuracy is not automatically better if validation metrics stagnate or decline. This risk is relevant for flexible models such as deep trees or neural networks, and for high-dimensional feature spaces with limited effective sample size.

This project mitigates overfitting using three complementary mechanisms. First, it uses stratified K-fold cross-validation data to reduce sensitivity to a single split. Second, it reserves an untouched held-out test set for final reporting. Third, it applies model-specific controls such as early stopping for MLP and complexity limits for tree-based methods.

Early stopping is particularly important in iterative learners like MLP: training is halted when validation performance stops improving, reducing the chance of

fitting late-stage noise. For distance-based or margin-based methods, proper scaling also contributes to stable generalisation because it prevents a few high-range features from dominating the optimization landscape.

Overall, overfitting control is not a single switch but a protocol-level property. Reliable results come from combining split strategy, hyperparameter discipline, scaling choices and final untouched testing.

4.4 Why Macro Metrics Matter

Accuracy alone can be misleading in multiclass tasks, even with roughly balanced classes, because it compresses all errors into a single number. Precision, recall, and F1 provide class-sensitive views of performance, and macro averaging gives each class equal weight.

For a single class, precision, recall, and F1-score can be defined as:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

where TP, FP, and FN represent true positives, false positives, and false negatives respectively. In a multiclass task, these values are computed separately for each class.

Macro F1 is then calculated as the unweighted mean of the per-class F1-scores:

$$Macro\ F1 = \frac{1}{K} \sum_{i=1}^{K} F1_i$$

where K is the number of classes and $F1_i$ is the F1-score for class i. This is why macro F1 is useful in this project: it prevents strong performance on one class from hiding weaker performance on another.

Macro precision is the arithmetic mean of per-class precision values; macro recall and macro F1 are defined similarly. This matters because a model can

achieve high global accuracy while underperforming on a specific class that may be scientifically important.
Macro metrics make this imbalance visible.

In this dissertation, macro metrics are the primary comparison criteria across models and dataset sizes. This choice aligns with the project goal of fair class treatment.
Confusion-matrix analysis complements macro metrics by identifying concrete error flows, such as exo_host misclassification as star, which is directly interpretable in domain terms.

The project reports multiple metrics together; accuracy (global correctness), macro precision (false-positive sensitivity per class), macro recall (false-negative sensitivity per class), and macro F1 (balance between precision and recall). Using all four avoids over-reliance on any single statistic and supports robust interpretation of trade-offs.

4.5 Model Families and Related Work Context

The model set includes a baseline plus classical supervised methods widely used in tabular classification.

Logistic Regression is a linear discriminative model and a strong baseline because it is fast, well-understood , and often competitive on clean, moderately separable data James *et al.* 2021. It benefits from standardised features for stable optimization and interpretable behaviour.

Random Forest is an ensemble of decision trees trained with randomness in sampling and feature selection, often delivering high performance and robustness on tabular data **Breiman2001** It captures nonlinear interactions and is less sensitive to monotonic feature scaling.

MLP (Multi-Layer Perceptron) provides nonlinear function approximation through stacked dense layers. It can learn complex boundaries but is more sensitive to scaling, architecture, and optimization settings. In this project, early stopping is used as a regularization control.

Extended comparison models include kNN, Decision tree, and SVM variants.

kNN is distance-based and therefore highly sensitive to feature scale; MinMax scaling is commonly effective in this setting. Decision tree provides interpretable hierarchical splits but can overfit without constraints. SVM methods (linear, RBF, polynomial kernels) are strong margin-based classifiers requiring careful scaling and kernel selection **Cortes and Vapnik**; **1995**.

Including both core and extended models improves methodological coverage: linear vs nonlinear, parametric vs instance-based, single-model vs ensemble, and low vs higher flexibility. This supports a more credible answer to research question of which model family is most suitable for the selected shared-feature multiclass setting.

4.6 Cross-validation and Held-Out Test Rationale

A central design choice is combining stratified K-fold cross-validation with a final held-out test set. These serve different purposes and should not be combined.

Stratified K-fold cross-validation (on the train + validation partition) provides a robust estimate of the model behaviour under repeated splits while preserving class ratios in each fold. This reduces variance relative to a single split and helps compare models more fairly. In this project, $K = 5$ is used as a practical balance between stability and runtime.

The held-out test set is not used during model selection. It is reserved for the final performance reporting only. This project protects optimistic bias caused by repeatedly tuning to validation results. Without an untouched test set, it is easy to overstate performance, especially when trying many models and preprocessing variants.

The two-level design is therefore methodologically important: CV supports selection and diagnostics; the held-out test supports the final generalisation claims. This protocol aligns with best practice in applied machine learning and strengthens the credibility of the reported results.

A further point is stratification. Because the task is multiclass, preserving class proportions during splitting avoids unstable fold composition and makes per-class metrics more comparable across runs. Together, stratification, K-fold CV, and held-out testing form the core of the project's overfitting-control strategy.

Table 1: Model summary used in this project.

Model	**Type**	**Needs scaling?**	**Strengths**	**Weaknesses**	**Why included**
Logistic Regression	Linear	Yes (Std)	Fast, interpretable	Limited nonlinearity	Core baseline
Random Forest	Ensemble trees	No	Strong tabular performance	Less interpretable	Core high performer
MLPClassifier	Neural net	Yes (Std)	Nonlinear capacity	Tuning-sensitive	Core neural comparison
kNN + MinMax	Instance-based	Yes (MinMax)	Strong local boundaries	Scale-sensitive, slower inference	Extended model, best test F1
Decision Tree	Single tree	No	Interpretable rules	Can overfit	Extended comparator
SVM (lin/rbf/poly)	Margin-based	Yes (Std)	Flexible kernels	Tuning/runtime sensitive	Extended comparison

Table 2: Overfitting mitigation techniques used in this project.

Technique	**Where used**	**Purpose**	**Example in this project**
Stratified split	Global protocol	Preserve class ratios in train/test	80% train+validation, 20% held-out test
Stratified K-fold CV	Model evaluation	Reduce split variance, robust comparison	$k = 5$ on train+validation partition
Held-out test set	Final reporting	Unbiased generalisation estimate	Test metrics reported only after model selection
Early stopping	MLP training	Limit overfitting during iterative learning	`early_stopping=True` in `MLPClassifier`
Model-specific scaling	Preprocessing	Stabilize optimization/distance behaviour	StandardScaler (LogReg/MLP/SVM), MinMaxScaler (kNN)
Complexity control	Tree/NN settings	Reduce excessive model flexibility	Constrained tree settings and fixed MLP architecture

5 Data Cleaning and Processing

This section describes the end-to-end preprocessing pipeline used to produce a fair multiclass dataset for star, exo_host, and quasar classification. The main objective of processing was to transform heterogeneous astronomy catalogues into a single, consistent feature space suitable for reproducible model comparison Gaia Collaboration 2016; Gaia Collaboration *et al.* 2023; Akeson *et al.* 2013. The workflow combined script-based cleaning, exoplanet enrichment with Gaia-like fields, column harmonization, and final notebook-level preprocessing for model-ready data NASA Exoplanet Archive 2026; ESA Gaia Archive 2023.

As shown in Figure 1, preprocessing followed a staged approach: dataset-specific cleaning, schema alignment, shared-feature extraction, missing-value handling, balanced sampling, and model-specific scaling. This design was chosen to separate domain/data-engineering tasks from model-training tasks and to reduce leakage risk Kohavi 1995; scikit-learn developers 2026b.

5.1 Data-specific cleaning scripts

Two core cleaning scripts were used for source-specific preprocessing:

- clean_exoplanets_full.py for exoplanet host data
- clean_stars_quasars_full.py for Gaia-based stars and quasars

These scripts standardized column selection, removed unusable records, and produced cleaned CSV outputs for downstream integration. The key principle was to clean each dataset in its own native schema first, and then align schemas only after quality control. This avoids silent data corruption from premature merges.

In the exoplanet dataset, cleaning focused on retaining host-star attributed to relevant astrometric/photometric comparison with Gaia-derived classes. In stars/quasars, cleaning ensured that positional and motion fields were consistently numeric and suitable for downstream joins and model input.

Table 3 summarizes the cleaning logic per dataset, including the operation type (column selection, renaming, numeric coercion, missing-data handling), rationale, and output file.

5.2 Exoplanet Gaia enrichment

A dedicated enrichment step (enrich_exo_csv_file.py) was applied to the exoplanet host to add or expose Gaia-like columns needed for multiclass compatibility NASA Exoplanet Archive 2026; ESA Gaia Archive 2023. Without enrichment, direct comparison between exoplanet hosts and Gaia stars/quasars would be limited by schema mismatch.

This step was essential because multiclass fairness depends on the information shared between all classes. If one class has unique high-signal columns unavailable to others, the model comparison becomes biased. The enrichment reduced this mismatch and enabled a final shared-feature modelling.

5.3 Column harmonization in notebook

Column harmonization was also important for reproducibility. If feature names are aligned manually but not documented, later experiments become difficult to verify because it is unclear whether two columns truly represent the same physical quantity. For example, `sy_plx` and `parallax` refer to the same type of astrometric measurement, but they originate from different catalogue naming conventions. Recording this mapping makes the preprocessing pipeline audit-able and reduces the chance of accidentally training models on inconsistent inputs.

The harmonization step also helped separate domain decisions from modelling decisions. Deciding that `sy_gaiamag` should align with `phot_g_mean_mag` is a data-engineering and domain-interpretation choice, while deciding whether to train kNN or Random Forest is a modelling choice. Keeping these steps explicit makes the final experiment easier to justify because the model comparison is built on a clearly defined feature space.

After script-level cleaning, the notebook performed explicit column alignment so that all three datasets shared identical feature names. The most important mappings were NASA Exoplanet Archive 2026:

- sy_plx -> parallax.
- sy_pmra -> pmra.
- sy_pmdec -. pmdec.

- sy_gaiamag -> phot_g_mean_mag.

This harmonization step made feature selection deterministic and avoided ambiguous references during model training. Table 4 documents this mapping and the reason for each rename.

5.4 Final multiclass preprocessing

The final multiclass pipeline in the notebook followed these steps:

1. Select shared features plus label column from each cleaned dataset.
2. Coerce selected feature columns to numeric (errors='coerce').
3. Handle remaining missing values in shared features.
4. Concatenate class datasets.
5. Build balanced subsets by sampling equal counts per class.
6. Build X and y for downstream stratified splitting/CV scikit-learn developers 2026g; scikit-learn developers 2026h.

The shared feature set used for all multiclass models was: ra, dec, parallax, pmra, pmdec, phot_g_mean_mag

Labels were fixed as: star, exo_host, quasar

This strict shared-feature policy was intentional: it ensured model differences reflected algorithmic behaviour, not unfair feature availability.

5.5 Missing values: imputation vs row removal

Missing data was treated with a conservative strategy focused on model comparability and reliability. For model input features, values were coerced to numeric, and rows with missing values in shared feature subset were removed in the final multiclass assembly stage James *et al.* 2021.

Median imputation is often preferred over mean imputation when distributors are skewed or outlier-prone because the median is more robust to extreme values James *et al.* 2021.

However, in this project, the final shared-feature modelling prioritized clean complete-case records to avoid introducing synthetic values that might blur class boundaries. Where rows were dropped, this was done only after feature harmonization and balanced sampling logic, so class fairness remained controlled.

The table 5 can show missing values before and after cleaning to make this decision transparent.

5.6 Scaling policy by model type

Preprocessing was model-aware rather than one-size-fits-all:

- StandardScaler: Logistic Regression, MLP, SVM.
- MinMaxScaler: kNN (and scaler-sensitivity comparison MLP).
- No scaling: Random Forest, Decision Tree.

This policy follows algorithm behaviour:

- Distance-based and gradient-based methods are sensitive to feature scale.
- Tree-based methods split on thresholds and are much less sensitive to monotonic feature scaling than distance-based or gradient-based methods.

A key clarification is the difference between normalization and standardization scikit-learn developers 2026f; scikit-learn developers 2026c:

- Normalization (MinMaxScaler) rescales values to a fixed range (typically 0 to 1)
- Standardization (StandardScaler) centers to mean 0 and scales to unit variance.

Both are valid, but suitability depends on the model mechanics.

5.7 Label encoding for MLP

For MLP-specific stability and compatibility, labels were encoded to integer class IDs via LabelEncoder scikit-learn developers 2026d; Pedregosa *et al.* 2011. This was applied inside the MLP evaluation flow only, while preserving original textual labels for reporting and confusion-matrix interpretation. This kept training numerically stable without losing semantic readability in analysis outputs.

5.8 Leakage prevention

Leakage prevention was enforced at protocol level Kohavi 1995; scikit-learn developers 2026g; scikit-learn developers 2026h; scikit-learn developers 2026b:

- Train/test split was enforced before final evaluation
- Cross-validation was stratified and executed on train+validation only.
- scaling was applied inside model pipelines, so transforms were fit only on training folds and then applied to validation/test data.
- Held-out test data remained untouched until final performance reporting.

This design prevented information from validation/test partitions from influencing model fitting or feature scaling, improving the credibility of generalisation claims.

5.9 Section Summary

Overall, preprocessing transformed heterogeneous astronomy sources into a consistent, fair, and model-ready multiclass dataset. Script-level cleaning ensured dataset integrity, exoplanet enrichment improved schema compatibility, notebook harmonization enforced shared features, and model-specific scaling supported fair algorithm comparison. Combined with stratified evaluation and leakage controls, this preprocessing strategy provided a robust foundation for the multiclass experiments reported in later sections.

Table 3: Cleaning-stage row counts per dataset.

Dataset	Rows (raw)	Rows (after script cleaning)	Rows (after shared-feature dropna)
Stars	50000	50000	50000
Quasars	50000	50000	50000
Exoplanet hosts	37871	37871	37871

Table 4: Column harmonization mapping used for multiclass alignment.

Dataset	**Original column**	**Aligned column**	**Reason**
Exoplanet hosts	`sy_plx`	`parallax`	Match Gaia-style naming for shared feature selection.
Exoplanet hosts	`sy_pmra`	`pmra`	Keep proper-motion axis naming consistent across classes.
Exoplanet hosts	`sy_pmdec`	`pmdec`	Keep proper-motion axis naming consistent across classes.
Exoplanet hosts	`sy_gaiamag`	`phot_g_mean_mag`	Align Gaia photometric brightness feature naming.
Stars/Quasars	`parallax`	`parallax`	Already aligned; no rename required.
Stars/Quasars	`pmra, pmdec, phot_g_mean_mag`	unchanged	Already Gaia-compatible schema.

Table 5: Missing-value handling summary before and after shared-feature filtering.

Dataset	Missing cells before	Missing cells after	Rows removed by dropna	Rows removed (%)
Stars	0	0	0	0.00
Quasars	0	0	0	0.00
Exoplanet hosts	0	0	0	0.00

6 Methodology and Experimental Design

This section defines the experimental protocol used to evaluate multiclass classification performance for `star`, `exo_host`, and `quasar`. The design prioritizes fairness, reproducibility, and reliable generalisation estimates by combining balanced subset construction, stratified cross-validation, and a held-out test set Kohavi 1995; scikit-learn developers 2026b.

6.1 Experimental Objective

The primary objective is to compare supervised models under a controlled shared-feature setting and determine which model family performs best for this multiclass astronomy task. The study also evaluates how performance changes with dataset scale and how preprocessing choices (especially scaling) affect model behaviour.

6.2 Dataset Construction for Experiments

After cleaning and harmonization, experiments use the shared feature set:

`ra, dec, parallax, pmra, pmdec, phot_g_mean_mag`

with class labels:

`star, exo_host, quasar`.

To ensure fair class treatment, balanced subsets are constructed by sampling equal numbers from each class. Three total subset sizes are used:

- 9,999
- 19,998
- 37,800

These correspond to equal per-class sample counts at each scale.

6.3 Train/Validation/Test Protocol

The evaluation protocol has two levels:

1. A stratified 80/20 split into train+validation and held-out test.

2. Stratified K-fold cross-validation ($K = 5$) on the train+validation partition.

Stratification preserves class proportions in all splits scikit-learn developers 2026g; scikit-learn developers 2026h. The held-out test set is not used during model selection and is evaluated only once after cross-validation. This reduces optimistic bias and improves credibility of final results Kohavi 1995.

6.4 Models Evaluated

A baseline and multiple supervised models are evaluated to compare linear, non-linear, tree-based, distance-based, and margin-based approaches:

- DummyClassifier (baseline)
- Logistic Regression
- Random Forest
- MLPClassifier
- kNN
- Decision Tree
- SVM (linear, RBF, polynomial)

6.5 Scaling and Encoding Policy

Preprocessing is model-specific:

- `StandardScaler`: Logistic Regression, MLP, SVM.
- `MinMaxScaler`: kNN (and MLP scaler-sensitivity comparison).
- No scaling: Random Forest, Decision Tree.

This policy follows known model behaviour for scale sensitivity scikit-learn developers 2026f; scikit-learn developers 2026c. For MLP experiments, labels are encoded to integer class IDs using `LabelEncoder` when needed for stable training workflow scikit-learn developers 2026d.

Table 6: Models and key experimental settings.

Model	Key settings	Role in study
DummyClassifier	default strategy baseline	Sanity-check reference
Logistic Regression	`max_iter=2000`	Linear core model
Random Forest	`n_estimators=300, class_weight=balanced_subsample`	Core tree ensemble
MLPClassifier	`hidden_layer_sizes=(128,64,32), max_iter=1000, early_stopping=True`	Core neural model
kNN	`n_neighbors=7, weights=distance`	Extended distance-based model
Decision Tree	`max_depth=12, min_samples_leaf=5`	Extended interpretable model
SVM (Linear)	`kernel=linear, C=1.0`	Extended margin-based model
SVM (RBF)	`kernel=rbf, C=1.0, gamma=scale`	Extended nonlinear kernel model
SVM (Poly)	`kernel=poly, degree=3, C=1.0, gamma=scale`	Extended polynomial kernel model

Table 7: Data-splitting and evaluation protocol.

Stage	Configuration	Purpose
Hold-out split	80% train+validation / 20% test (stratified)	Final unbiased test set
Cross-validation	Stratified 5-fold on train+validation	Robust model comparison
Final fit	Train on full train+validation	Maximize training data before test
Final evaluation	Single run on held-out test	Report test metrics

6.6 Evaluation Metrics

Model performance is measured with:

- Accuracy
- Macro Precision
- Macro Recall
- Macro F1

Macro averaging gives equal weight to each class and supports fair multiclass assessment Sokolova and Lapalme 2009; scikit-learn developers 2026e. Confusion matrices are used for the implementation of class-level error scikit-learn developers 2026a.

6.7 Leakage Prevention and Reproducibility

Several controls are used to prevent leakage and improve reproducibility:

- Train/test split performed prior final evaluation.
- Cross-validation restricted to train+validation only.
- Scaling applied inside pipelines so that transformers are fit on training folds only.
- Fixed random seeds (`random_state=42`) for deterministic sampling/splitting.
- Consistent shared-feature schema across all models.

6.8 Methodology Summary

The methodology combines balanced multiclass subset construction, stratified 5-fold cross-validation, and held-out test evaluation under model-specific preprocessing rules. This protocol supports fair model comparison and robust generalisation claims for the selected astronomy classification task.

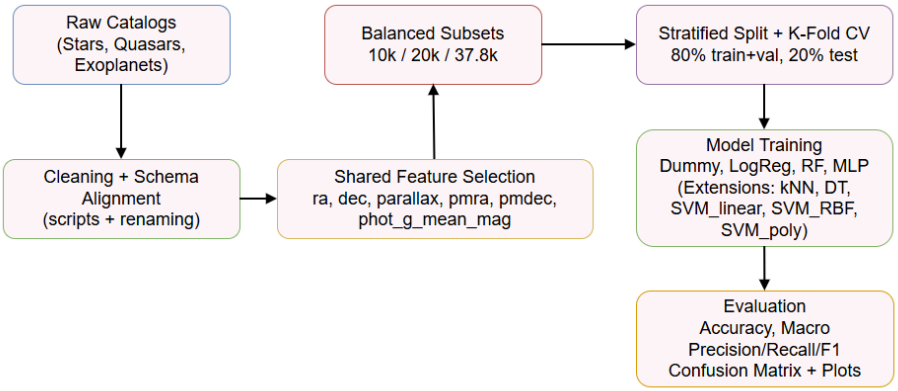

Figure 1: Experimental workflow: cleaned data → balanced subsets → stratified split → stratified 5-fold CV → final held-out test evaluation.

7 Implementation

This section explains how the multiclass experiment was implemented in practice, including notebook structure, reusable evaluation functions, result aggregation, plotting, and runtime considerations. The implementation was developed in Jupyter Notebook to support iterative experimentation and reproducible analysis.

7.1 Notebook structure

The multiclass notebook is organized as a sequential workflow from data preparation to the final evaluation output. The high-level structure is:

1. Imports and configuration.
2. Data loading for cleaned stars, exoplanet hosts, and quasars.
3. Schema alignment and shared-feature definition.
4. Balanced subset construction at multiple dataset sizes.
5. Reusable evaluation function definitions.
6. Model execution loops.
7. Result aggregation into dataframes.

8. Plot generation and saving.

9. Confusion matrix and class-level error analysis.

This structure separates preprocessing, modelling, and reporting outputs, reduced duplication, and making the notebook easier to maintain.

7.2 Reusable Evaluation Functions

A key implementation choice was to define reusable functions for each model family and one generalized function for additional models. Core functions include:

- `eval_size_logreg(...)`
- `eval_size_rf(...)`
- `eval_size_mlp(...)`
- `eval_model_multiclass(...)` (generic for kNN/DT/SVM variants)

Each function follows the same evaluation pattern:

Build a balanced subset for the requested total size.

Apply train+validation/test split (stratified).

Run stratified 5-fold cross-validation on train +validation.

Fit on full train+validation.

Evaluate once on held-out test.

Return standardized metrics in dictionary format.

This implementation pattern ensures metric compatibility across all model families.

Label assignment and Balanced sampling

The Class labels were explicitly assigned as `star`, `exo_host`, and `quasar`. Balanced subset construction was implemented by sampling equal rows per class for each target total size. This ensured fairness in multiclass comparison and prevented class-frequency bias in model ranking. A short implementation pattern is shown below:

```
n_per = min(n_total // 3, len(stars), len(exo), len(quasars))
df = pd.concat([
    stars.sample(n=n_per, random_state=42),
    exo.sample(n=n_per, random_state=42),
    quasars.sample(n=n_per, random_state=42)
], ignore_index=True)
```

7.3 CV/Test Evaluation Skeleton

All core model functions followed the same skeleton for robust evaluation:

```
# 80/20 hold-out
X_trainval, X_test, y_trainval, y_test = train_test_split(..., stratify=y)

# stratified K-fold on train+val
for train_idx, val_idx in skf.split(X_trainval, y_trainval):
    clf.fit(X_trainval[train_idx], y_trainval[train_idx])
    pred = clf.predict(X_trainval[val_idx])
    # compute fold metrics

# final test
clf.fit(X_trainval, y_trainval)
test_pred = clf.predict(X_test)
# compute test metrics
```

This keeps model selection (CV) and final reporting (held-out test) clearly separated.

7.4 Result Aggregation and Plotting

After each model run, metric dictionaries were converted to dataframes and tagged with a model name. These were concatenated into a single comparison dataframe for plotting and table generation.

Typical aggregation flow:

- Run model functions for each size.
- Convert to `pd.DataFrame`.

- Add `model` column.
- Concatenate into a unified plotting/report dataframe.

Plots were generalized using `seaborn` and `matplotlib`, including:

- CV metrics vs dataset size plots.
- Test metric vs dataset size plots
- All-model comparison plot(s).
- Confusion matrix for selected best model(s).

7.5 Saving Outputs

Plot outputs were saved to standardized project folders under `Main_plts/`. Consistent output naming was used to simplify report integration and reduce file confusion.

Example structure:

- `Main_plts/multiclass_stars_exo_quasars`
- `Main_plts/supervidesed_models/`
- `Main_plts/mlp_scaler_comparison/`
- `Main_plts/mlp_scaler_comparison/`

7.6 Runtime Considerations

Runtime varied by algorithm family:

- Logistic Regression, Random Forest, Decision Tree, and kNN were practical across all tested sizes.
- MLP was slower but still feasible with early stopping.
- SVM with RBF and polynomial kernels and higher runtime, especially at larger subsets sizes.

To keep experiments tractable, heavier SVM variants were optionally run on smaller subset sizes first (e.g., 10k and 20k), then extended only if runtime permitted.

Software Environment

Implementation was conducted in Python using:

- `pandas` and `numpy` (data manipulation),
- `scikit-learn` (models, CV, metrics, pipelines),
- `matplotlib` and `seaborn` (visualization).

Table 8: Software stack and execution environment.

Component	**Version / Spec**	**Purpose**
Python	3.13.0	Core runtime
Operating System	Windows 11 (10.0.26200)	Execution environment
pandas	2.3.3	Data loading and transformation
NumPy	2.3.5	Numerical operations
scikit-learn	1.7.2	Models, cross-validation, metrics, pipelines
matplotlib	3.10.7	Plot generation
seaborn	0.13.2	Statistical visualization
astroquery	0.4.11	Astronomical data access/enrichment support
Jupyter Notebook	Used via notebook workflow	Interactive experiment environment
CPU	AMD ryzen 9 7900X3D	Available hardware

7.7 Implementation Summary

The implementation is modular, reproducible, and aligned with the methodological goals of fair multiclass comparison. Reusable evaluation functions, consistent split logic, and standardized output handling made it possible to compare multiple model families under the same protocol and report results in a structured way.

8 Results

This chapter reports the experimental results for the supervised classification experiments. The results are presented using the evaluation protocol described in

the Methodology chapter: stratified 5-fold cross-validation on the train+validation partition, followed by final evaluation on an untouched held-out test Kohavi 1995; scikit-learn developers 2026b. Performance is reported using accuracy, macro precision, macro recall, and macro F1. Macro-averaged metrics are emphasized because they give equal importance to each class in the multiclass setting Sokolova and Lapalme 2009; scikit-learn developers 2026e. The chapter first summarized the preliminary binary experiments, then focuses on the main multiclass task involving `star`, `exo_host`, and `quasar`.

8.1 Preliminary Binary experiments

Before the final multiclass experiment, binary classification experiments were carried out as preliminary validation of the cleaning and modelling pipeline. These binary experiments tested whether the processing workflow, feature extraction, model fitting, metric reporting, and plotting logic behaved correctly on simpler two-class tasks before being extended on the final three-class setting.

The preliminary binary tasks were not the main contribution of this dissertation. However, they were useful for checking that the cleaned datasets could support supervised classification and the modelling code produced stable outputs. They also provided early evidence that the selected features carried class-discriminative information. For this reason, the binary experiments are included as pipeline validation rather than analyzed in the same depth as the final multiclass experiment.

Table 9 summarizes the binary experiments. The results are reported briefly because the main research objective is multiclass classification across `star`, `exo_host`, and `quasar`.

8.2 Dummy baseline

A DummyClassifier baseline was included as a simple reference point for the multiclass experiment. The baseline used the `most_frequent` strategy, meaning it always predicted the majority class in the training split. Since the multiclass subset was balanced across `star`, `exo_host`, and `quasar`, this strategy is expected to achieve approximately one-third accuracy. The purpose of this baseline was to confirm that the supervised models provided significant improvement over a trivial prediction rule.

The dummy baseline also provides useful context for interpreting the high

Table 9: Summary of preliminary binary classification experiments.

Binary task	Dataset version / size	Evaluation protocol	Best model	Mean F1
Asteroids vs Comets	Legacy/sample data, approx. 7,000 rows	Repeated stratified shuffle split	RF	0.9921
Host vs Non-host Stars	Legacy/sample data, approx. 10,000 rows	Repeated stratified shuffle split	RF	0.9984
Quasars vs Stars	Small exploratory sample, approx. 50, 000 rows	Repeated stratified shuffle split	RF	0.9488

scores achieved by the supervised models. Because the dataset is balanced across three classes, always predicting the most frequent class gives approximately 0.3333 accuracy. The macro F1 score is lower because two of the three classes receive no correct predictions under this strategy. This confirms that macro F1 is more informative than accuracy for detecting poor class-level behaviour.

The large gap between the dummy baseline and the trained models shows that the selected features contain meaningful discriminative signal. If the supervised models had only slightly exceeded the dummy baseline, this would suggest weak feature quality or an unsuitable modelling setup. Instead, the trained models achieved substantially higher results, supporting the validity of the shared-feature classification approach.

Table 10: DummyClassifier baseline result using the `most_frequent` strategy.

Strategy	Mean Accuracy	Mean Macro F1
`most_frequent`	0.3333	0.1667

This baseline was evaluated using repeated stratified shuffle splits and is used only as a chance-level reference, not as part of the final model-ranking procedure.

8.3 Main Multiclass Model Results

The main multiclass comparison evaluated Logistic Regression, Random Forest, and MLPClassifier across three balanced subsets sizes: 9,999, 19,998, and 37,800

samples. These models represent three different model families: A linear classifier, a tree-based ensemble, and a neural-network classifier. All models were evaluated under the same split strategy and metric set.

Table 11 reports across-validation and held-out test results for the main multiclass models. Logistic Regression produced the lowest performance among the three main models, but its results were stable across dataset sizes. Its held-out test macro F1 remained close to 0.98, indicating that a linear model captured a substantial amount of the class structure but does not fully exploit nonlinear relationships in the feature space.

Random Forest produced the strongest results among the three main models. Its cross-validation macro F1 improved from 0.9977 at 9,999 samples to 0.9993 at 37,800 samples. However, despite this improvement, it remained below Random Forest in cross-validation performance and below kNN in final held-out test performance.

Table 11: Main multiclass model results across balanced dataset sizes.

Model	**Size**	**CV Acc**	**CV Macro F1**	**Test Acc**	**Test Macro F1**
Logistic Regression	9,999	0.9769	0.9769	0.9800	0.9800
Logistic Regression	19,998	0.9790	0.9790	0.9775	0.9775
Logistic Regression	37,800	0.9794	0.9794	0.9802	0.9802
Random Forest	9,999	0.9977	0.9977	0.9970	0.9970
Random Forest	19,998	0.9990	0.9990	0.9990	0.9990
Random Forest	37,800	0.9993	0.9993	0.9996	0.9996
MLPClassifier	9,999	0.9899	0.9899	0.9845	0.9845
MLPClassifier	19,998	0.9949	0.9949	0.9965	0.9965
MLPClassifier	37,800	0.9963	0.9963	0.9978	0.9978

Figures 2–5 visualize the same trends across dataset size. The main pattern is that nonlinear models outperform Logistic Regression, and both Random Forest and MLPClassifier benefit from increased sample size. Random Forest remains the strongest of the initial core models.

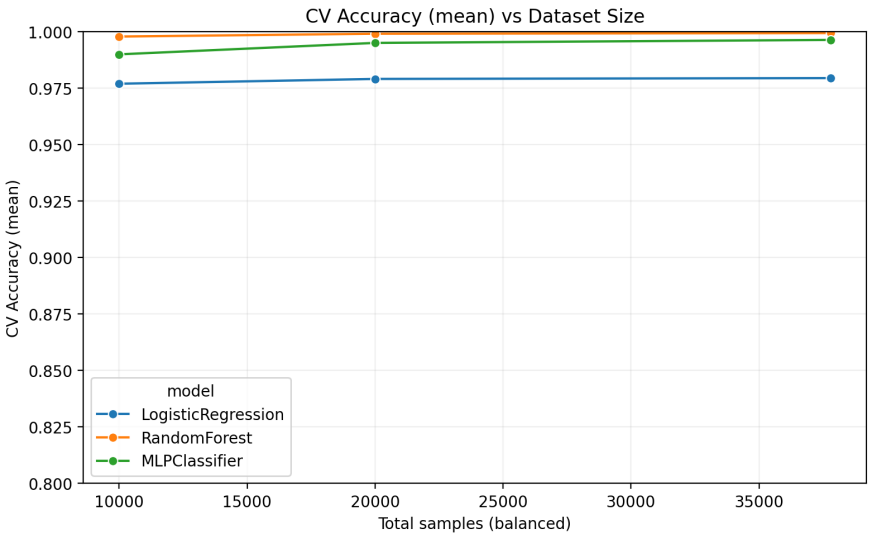

Figure 2: Cross-validation accuracy versus dataset size for the main multiclass models.

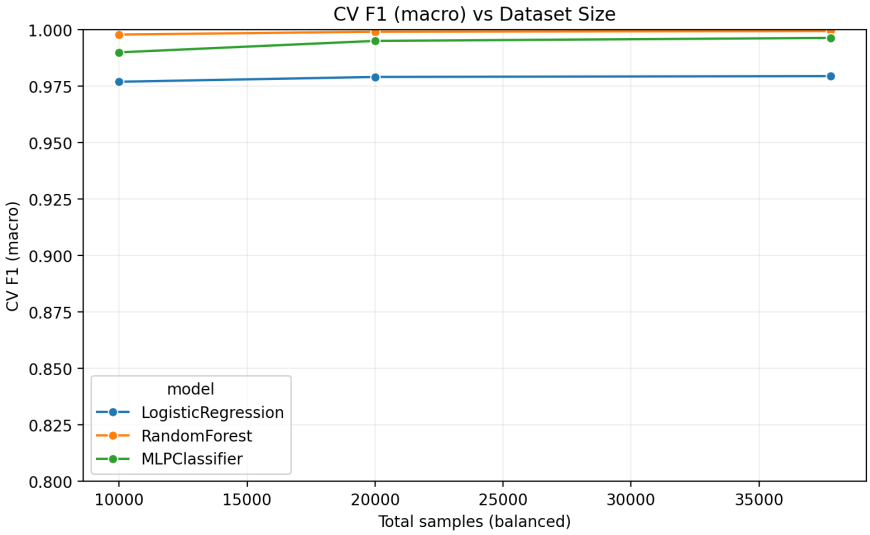

Figure 3: Cross-validation macro F1 versus dataset size for the main multiclass models.

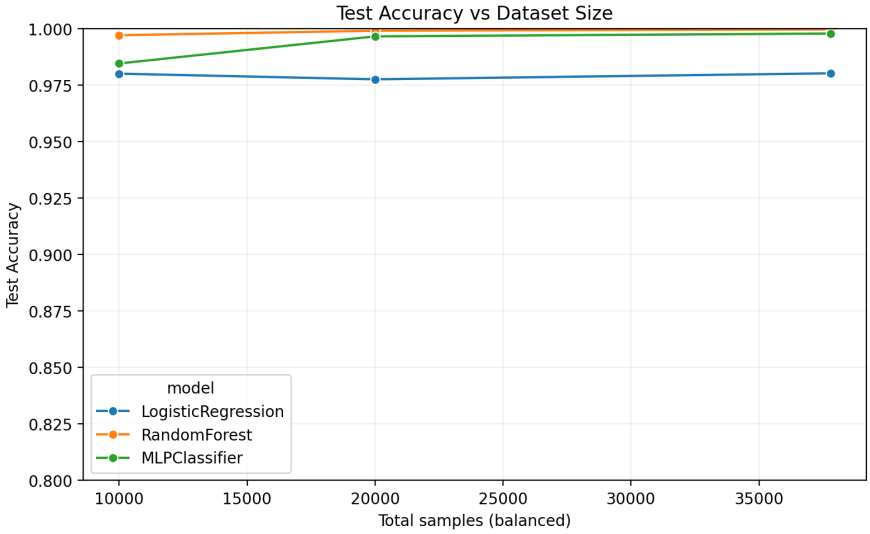

Figure 4: Held-out test accuracy versus dataset size for the main multiclass models.

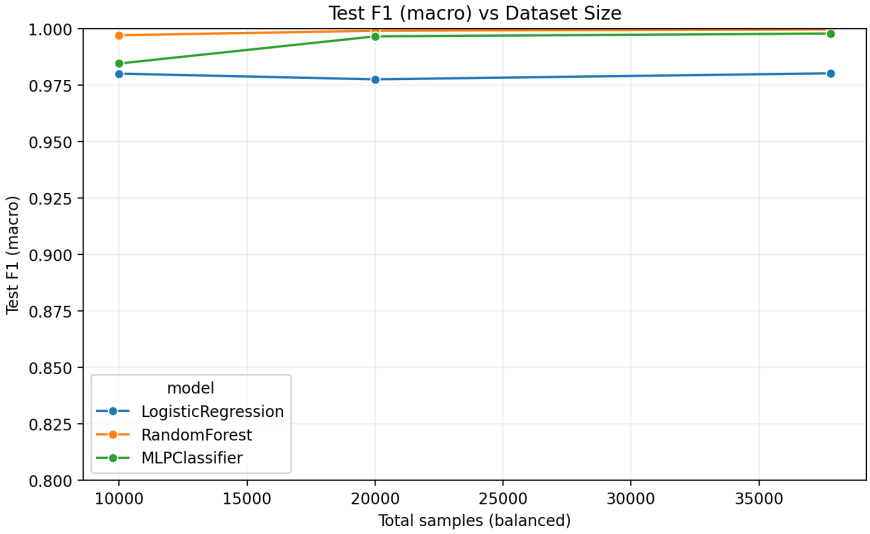

Figure 5: Held-out test macro F1 versus dataset size for the main multiclass models.

8.4 Extended Model Results

Additional models were evaluated to broaden the comparison beyond the initial core models. These included kNN with MinMax scaling, Decision Tree, and SVM variants with linear, RBF, and polynomial kernels. The purpose of this extension was to test whether other models families could outperform the core models under the same shared-feature and evaluation constraints. Table 12 summarizes the extended-model results. kNN with MinMax scaling achieved the strongest held-out test macro F1 on the largest balanced subset, reaching 0.9997. This was slightly higher than the Random Forest held-out test macro F1 of 0.9996. Decision Tree also performed strongly, reaching a held-out test macro F1 of 0.9984 at the largest dataset size, but it remained below both kNN and Random Forest.

The SVM variants models performed worse than the top-performing tree-based and distance-based models in this setting. Linear SVM achieved a held-out test macro F1 of 0.9847 at 37,800 samples, similar to Logistic Regression. RBF and polynomial SVMs were more computationally expensive, and the nonlinear SVM variants were therefore evaluated only on smaller subset sizes where appropriate.

The extended model results show that the best-performing models were very close numerically. kNN with MinMax scaling achieved the highest held-out test macro F1, but the difference compared with Random Forest was extremely small. This means the result should not be overstated as a large practical advantage. Instead, the evidence suggests that both models are strong candidates for this feature space, with kNN slightly ahead on the final held-out split and Random Forest stronger in cross-validation.

The SVM variants were useful as additional comparisons, but they were less practical at larger dataset sizes. Linear SVM behaved similarly to Logistic Regression, which is expected because both methods learn broadly linear decision boundaries. RBF and polynomial SVMs can model nonlinear structure, but they are computationally more expensive. Since their smaller-size results did not exceed kNN or Random Forest, extending every nonlinear SVM run to the largest dataset size was not necessary for the main conclusion.

Figures 6 and 7 compare all the evaluated models using macro F1. These figures show the strongest models are clustered very closely near the top of the scale. The small numerical difference between Random Forest and kNN should therefore be interpreted carefully: kNN achieved the highest final held-out test

Table 12: Extended multiclass model results.

Model	Size	CV Acc	CV Macro F1	Test Acc	Test Macro F1
kNN + MinMax	9,999	0.9957	0.9957	0.9970	0.9970
kNN + MinMax	19,998	0.9976	0.9976	0.9993	0.9993
kNN + MinMax	37,800	0.9987	0.9987	0.9997	0.9997
Decision Tree	9,999	0.9930	0.9930	0.9950	0.9950
Decision Tree	19,998	0.9959	0.9959	0.9960	0.9960
Decision Tree	37,800	0.9979	0.9979	0.9984	0.9984
SVM Linear	9,999	0.9826	0.9826	0.9835	0.9835
SVM Linear	19,998	0.9845	0.9845	0.9840	0.9840
SVM Linear	37,800	0.9837	0.9837	0.9847	0.9847
SVM RBF	9,999	0.9891	0.9891	0.9835	0.9835
SVM RBF	19,998	0.9914	0.9914	0.9905	0.9905
SVM Poly	9,999	0.9741	0.9742	0.9685	0.9686
SVM Poly	19,998	0.9798	0.9798	0.9788	0.9788

score, while Random Forest achieved the strongest average cross-validation score.

8.5 MLP Scaler Comparison

A scaler comparison was conducted for MLPClassifier to test whether StandardScaler or MinMaxScaler produced strongest performance under otherwise similar model settings. This comparison was included because neural networks are sensitive to feature scaling and optimization stability.

Table 13 reports the MLP comparison. The results should be interpreted as a preprocessing sensitivity check rather than as a separate model family. The purpose was to confirm whether the selected scaling methods materially affected the MLP results.

Figures 8 shows that StandardScaler performed slightly better at 9,999 and 19,998 samples, while MinMaxScaler produced the highest held-out test macro F1 at 37,800 samples. The differences were small overall, indicating that the MLP performance was relatively stable under both scaling choices.

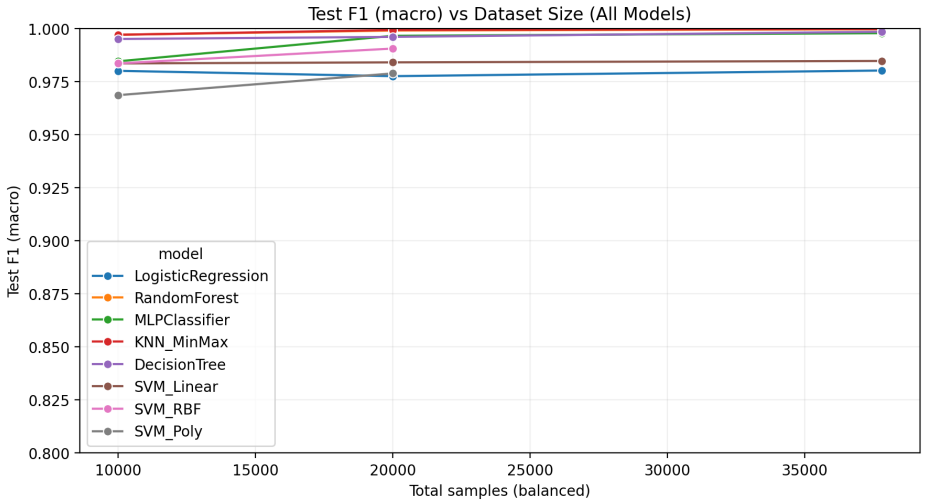

Figure 6: Held-out test macro F1 versus dataset size for all evaluated models.

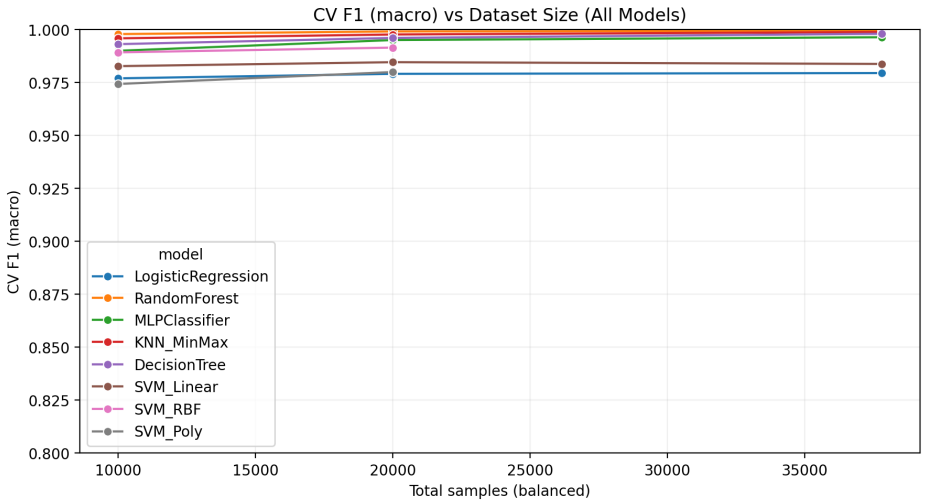

Figure 7: Cross-validation macro F1 versus dataset size for all evaluated models.

Table 13: MLP scaler comparison.

Scaler	Size	CV Acc	CV Macro F1	Test Acc	Test Macro F1
StandardScaler	9,999	0.9899	0.9899	0.9845	0.9845
StandardScaler	19,998	0.9949	0.9949	0.9965	0.9965
StandardScaler	37,800	0.9963	0.9963	0.9978	0.9978
MinMaxScaler	9,999	0.9887	0.9887	0.9830	0.9830
MinMaxScaler	19,998	0.9931	0.9931	0.9930	0.9930
MinMaxScaler	37,800	0.9964	0.9964	0.9988	0.9988

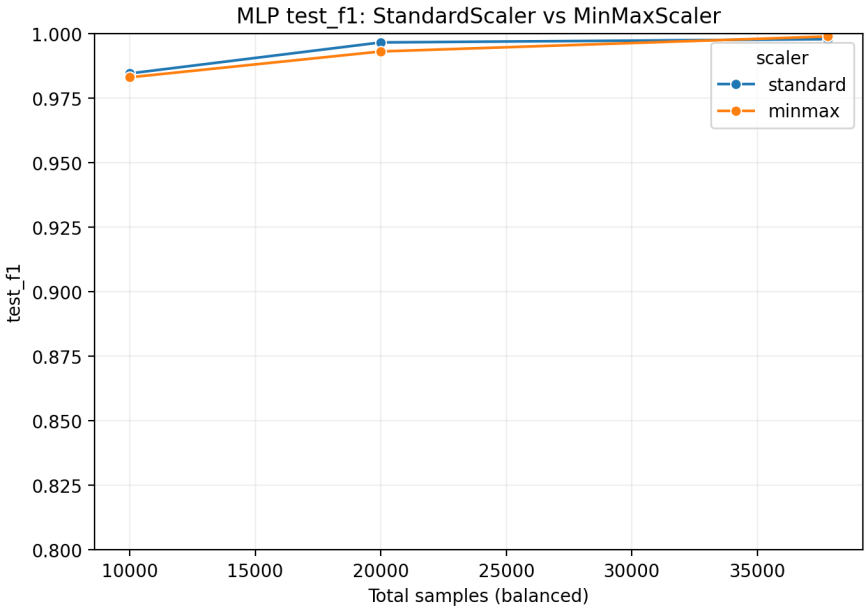

Figure 8: MLP held-out test macro F1 comparison between StandardScaler and MinMaxScaler.

8.6 Confusion Matrix and Error Analysis

The best held-out test result was achieved by kNN with MinMax scaling on the largest balanced subset. Figure 9 shows the confusion matrix for this model. The matrix shows near-perfect class separation, with only two misclassifications in the held-out test set. Both errors were `exo_host` samples predicted as `star`. The `quasar` class had no errors in this final evaluation.

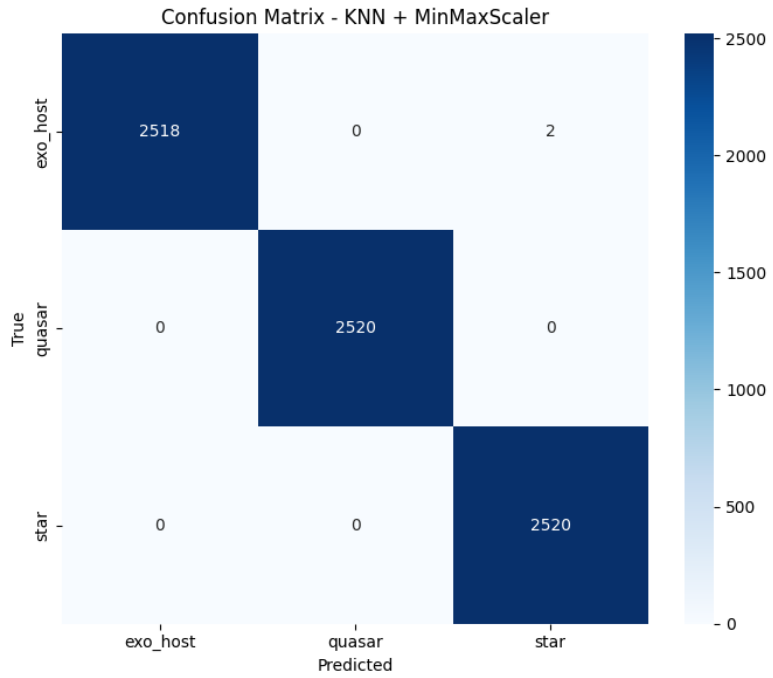

Figure 9: Confusion matrix for kNN + MinMaxScaler on the largest balanced subset.

Table 14 reports class-wise precision, recall, F1-score, and support for the same model. The support value of 2,520 per class reflects the 20% held-out test split the largest balanced dataset of 37,800 samples. The total test set therefore contained 7560 samples.

Table 14: Classification report for kNN + MinMaxScaler on the held-out test set.

Class	**Precision**	**Recall**	**F1-score**	**Support**
`exo_host`	1.0000	0.9992	0.9996	2520
`quasar`	1.0000	1.0000	1.0000	2520
`star`	0.9992	1.0000	0.9996	2520
Accuracy		0.9997		7560
Macro avg	0.9997	0.9997	0.9997	7560
Weighted avg	0.9997	0.9997	0.9997	7560

For comparison, Random Forest also achieved very strong performance and produced the strongest cross-validation macro F1 on the largest dataset. This distinction is important because the two evaluation answer slightly different questions. Cross-validation reflects average performance across folds, while the held-out test set gives final performance on one untouched split. In this experiment, kNN was the best final held-out model, while Random Forest was the strongest cross-validation model.

8.7 Results Summary

Overall, the results show that all trained models substantially outperformed the dummy baseline.
Logistic Regression provided a stable but lower-performing linear benchmark. Random Forest achieved the strongest cross-validation macro F1 and very strong held-out test performance. kNN with MinMax scaling achieved the strongest held-out test macro F1 on the largest balanced subset.
MLPClassifier improved with dataset size but did not exceed the best classical models.

The confusion matrix confirmed that final errors were extremely limited. For the best held-out model, only two `exo_host` samples were predicted as `star`, while the `quasar` class was classified perfectly. These results indicate that the selected shared Gaia-like features provide strong separation between the three classes under the chosen preprocessing and evaluation protocol.

The results also show that model ranking depends slightly on which evaluation view is prioritized. Random Forest achieved the strongest cross-validation performance, suggesting that it was highly stable across folds. kNN with MinMax scaling achieved the strongest final held-out test macro F1, which makes it the best final test model in this experiment. Since both scores are extremely close, the most careful conclusion is that both models are strong, with kNN slightly ahead on the final test split and Random Forest slightly stronger in cross-validation stability.

This distinction is useful for reporting because it avoids overstating a very small numerical difference. In practical terms, the best model choice may depend on whether the priority is the final held-out test score, cross-validation stability, interpret-ability, prediction-time cost, or robustness to future data. The dissertation therefore reports both views rather than reducing the conclusion to a single

simplistic ranking.

9 Discussion

This chapter interprets the results in the previous section and links them back to the project research questions. The discussion focuses on model behaviour, dataset-size effects, cross-validation versus held-out test consistency, scaling effects, fairness of comparison, and the practical limitations of the conclusions.

9.1 Overall Performance Trends

The results show that the selected shared Gaia-like features were highly effective for separating `star`, `exo_host`, and `quasar`. All trained models substantially outperformed the DummyClassifier baseline, confirming that the classification results were not caused simply by class balance or chance-level prediction.

The strongest models were kNN with MinMax scaling and Random Forest. kNN achieved the highest held-out test macro F1 on the largest available dataset, while Random Forest achieved the strongest cross-validation macro F1. This distinction is important because of the held-out test behaviour across multiple folds. Since the numerical gap between the two models was very small, the results should be interpreted as evidence that both models are highly suitable for this dataset, rather than as evidence of large practical difference.

Logistic Regression and linear SVM performed lower than the nonlinear models, although their scores remained high in absolute terms. This suggests that the three classes are partly linear separable, but that nonlinear decision boundaries better capture the structure in the selected features.
MLPClassifier also performed strongly and improved with dataset size, but it did not exceed kNN or Random Forest in the final comparison.

9.2 Why kNN and Random Forest Performed Strongly

kNN with MinMax scaling likely performed strongly because the selected feature space contains meaningful local neighborhood structure. Since kNN predicts based on nearby samples, it can be effective when objects from the same class cluster closely after appropriate scaling. MinMax scaling was especially relevant

because kNN is distance-based; without scaling, features with larger numeric ranges could dominate the distance calculationCover and Hart 1967; scikit-learn developers 2026c.

Random Forest likely performed strongly because it can capture nonlinear feature interactions without requiring explicit feature engineeringBreiman 2001 . The model combines many decision trees, reducing the instability of a single tree and improving robustness. It is also less sensitive to monotonic feature scaling, making it well suited to heterogeneous tabular features. Its strong cross-validation performance suggests that it generalized consistently across folds.

The difference between kNN and Random Forest also has a practical interpretation. kNN is simple at training time because it stores the training examples and makes predictions by comparing new samples with nearby points. This can work very well when the feature space is clean and scaled properly, as in this project. However, kNN can become slower at prediction time as the dataset grows, because each new prediction depends on distance comparisons with stored examples.

Random Forest has different practical strengths. Training is more expensive than a single decision tree, but prediction is usually efficient once the ensemble has been fitted. It also provides a more robust model than a single tree because many trees are averaged together. For this reason, even though kNN achieved the highest held-out test macro F1, Random Forest remains an attractive model for deployment-style use where stability, robustness, and predictable inference are important.

The comparison between kNN and Random Forest is therefore not simply a question of one model being universally better. kNN produced the best final held-out test macro F1, while Random Forest showed slightly average cross-validation stability. In practice, this means kNN is the reported best final test model, while Random Forest remains a highly reliable alternative.

9.3 Effect of Dataset Size

Increasing dataset generally improved model performance, especially for Random Forest, MLPClassifier, kNN and Decision Tree. This trend indicates that the additional training data helped models learn more stable class boundaries. However, the gains diminished at larger sizes because several models were already close to ceiling performance by 19,998 samples.

This diminishing return is important. Moving from 9,999 to 19,998 samples produced visible improvements for several models, but the improvement from 19,998 to 37,800 was smaller. This suggests that, under the selected feature set, the classification problem becomes highly separable once enough balanced examples are available. Additional data still helps, but the marginal benefit decreases as performance approached near-perfect levels.

The dataset-size trend also helps evaluate whether the experiment was too small or whether the results were unstable. If performance had changed unpredictably between subset sizes, it would suggest that the conclusions were highly dependent on sampling noise. Instead, the strongest models improved or remained stable as the dataset size increased. This gives more confidence that the results are not only a consequence of a lucky split at one specific sample size.

At the same time, the near-ceiling performance means that further improvements become difficult to interpret. When models already achieve macro F1 scores above 0.99, very small differences may reflect split variation rather than meaningful superiority. This is why the discussion treats the gap between kNN and Random Forest cautiously. The important finding is not only that one model is marginally higher, but that the selected preprocessing and shared features allow several classical models to classify the three classes with very high reliability.

Logistic Regression showed the smallest improvements with dataset size. This is consistent with a model whose capacity is limited by its linear decision boundary rather than by insufficient training examples. By contrast, MLPClassifier and Random Forest benefited more from additional data because their flexible decision functions can exploit more examples.

9.4 Overfitting and Generalisation

The use of stratified cross-validation and a held-out test set allowed generalisation to be assessed more carefully than with a single split alone. Overall, cross-validation and test results were consistent for the strongest models. Random Forest and kNN had very small differences between CV and held-out test macro F1, suggesting limited overfitting under the final protocol.

MLPClassifier results also suggest reasonable generalisation, especially at larger dataset sizes.

Early stopping helped reduce the risk of overfitting during training, while scaling supported stable optimization. However, MLP remained more sensitive to processing and configuration than Random Forest or kNN.

The Decision Tree performed strongly but remained below Random Forest, which is expected because a single tree is more vulnerable to overfitting and instability. Random Forest reduces this risk by aggregating many trees. The results therefore support the value of ensemble methods for structured tabular data.

9.5 Effect of scaling on MLP

The MLP scaler comparison showed that scaling choice affected performance but did not completely change the model ranking. StandardScaler performed slightly better at smaller dataset sizes, while MinMaxscaler produced the highest MLP held-out test macro F1 at the largest dataset size. The differences were relatively small, showing that the MLP was fairly robust once features were consistently scaled.

This result is important because it supports the decision to test preprocessing choices rather that assume one scaler is always optimal. It also reinforces the broader point that scaling should be model-specific. kNN benefited from MinMax scaling because of its distance-based prediction rule, while Logistic Regression, MLP, and SVM were evaluated with stratifiedScaler because of optimization and margin sensitivity.

9.6 Fairness of the Model Comparison

The results show that classical machine learning models are highly effective for this type of structured astronomical classification task. In particular, kNN and Random Forest achieved near-perfect performance on the selected multiclass problem. This indicates that deep learning is not automatically necessary when the data are tabular, clean, and well aligned.

The fairness of the comparison depends heavily on the shared-feature design. If one dataset contained a class-specific identifier or a feature only available for that class, a model could learn the origin of the row rather than the astrophysical distinction between classes. This would produce high metrics but weak scientific

validity. Restricting the model input to features available across all classes reduces this risk and makes the comparison more defensible.

Balanced sampling also contributes to fairness. Without balancing, the larger classes could dominate the training objective and the evaluation metrics. In that situation, a model might perform well for stars and quasars while being weaker for exoplanet hosts. Equal class counts ensure that each class contributes equally to training and that macro metrics reflect class-level performance rather than class frequency.

Runtime is also relevant to a fair practical comparison. Logistic Regression and Decision Tree were relatively lightweight. Random Forest and kNN were practical for the tested sizes. SVM with RBF and polynomial kernels was more computationally demanding, which limited evaluation at the largest size. This creates a performance/runtime trade-off: high accuracy alone is not the only practical consideration.

There are also fairness-related limitations. The experiment used only three classes and only shared Gaia-like features. The model may perform differently if additional object classes are included, if noisier raw data are used, or if external validation data are introduced. Near-perfect performance also raises the need for careful leakage checks and dataset-overlap validation, especially where exoplanet hosts and general stars may overlap by Gaia identifiers.

9.7 Practical Meaning of the Results

The very high scores should be interpreted in the context of the project design. The aim was not to prove that all celestial object classification is solved, but to test whether a controlled multiclass pipeline could separate three selected object categories using shared Gaia-like tabular features. Under this controlled setting, the results are strong: the best models made very few errors, and the class-wise metrics were consistent across precision, recall, and F1-score.

The practical meaning is that a carefully cleaned and aligned catalogue can support highly accurate automated classification without requiring an image-based deep learning system. This is important because tabular catalogue models are often easier to train, faster to evaluate, and simpler to inspect than large neural models. For a project focused on reproducible machine learning, this is a meaningful out-

come. It shows that much of the challenge is not only choosing the most advanced algorithm, but building a reliable data preparation and evaluation pipeline.

The results also show why it is useful to compare several model families. Logistic Regression provided a stable linear benchmark, but it did not match the nonlinear models. MLPClassifier improved with additional data, but it required scaling and early stopping to behave reliably. Random Forest was consistently strong across cross-validation folds, while kNN with MinMax scaling achieved the best final held-out test result. This pattern gives a more complete picture than reporting a single model alone.

From a practical deployment perspective, the small difference between kNN and Random Forest would need further testing before making a final operational choice. kNN achieved the best held-out test macro F1, but Random Forest may be preferable where prediction-time efficiency, robustness, and easier deployment are priorities. This is why the dissertation reports both the best held-out model and the strongest cross-validation model rather than presenting the ranking as absolute.

The confusion matrix also gives a practical interpretation of the remaining error. The final kNN model produced only two errors in the held-out test set, both involving `exo_host` samples predicted as `star`. This is understandable because exoplanet hosts are still stars physically, so the distinction is more subtle than separating stars from quasars. This supports the value of class-level analysis: the overall score is very high, but the remaining mistakes still reveal where the boundary is most difficult.

Overall, the results suggest that the project pipeline is effective for the selected problem, but they should be treated as evidence within a controlled experimental scope. The next step would be testing whether the same approach remains reliable when the class set is expanded, the features are richer, or an independent external catalogue is used.

9.8 Research Questions Revised

The first research question asked which classical machine learning model performs best on the selected shared Gaia-like features. The answer depends on the evaluation view: kNN with MinMax scaling achieved the best held-out test macro F1, while Random Forest achieved the strongest cross-validation F1.

The second research question asked how performance changes with dataset size. Overall, performance improved as dataset size increased, especially for non-linear models, but improvements diminished at larger sizes as models approached ceiling performance.

The third question asked whether scaling choice affects MLP performance. The scaler comparison showed small but measurable differences, with StandardScaler stronger at smaller sizes and MinMaxScaler stronger on the largest held-out test result.

Table 15: Summary of key discussion findings.

Finding	**Supporting result/figure**	**Interpretation**	**Limitation**
kNN achieved the best held-out result	kNN + MinMax reached the highest test macro F1	Local neighborhood structure is highly informative after MinMax scaling	Based on one final held-out split
Random Forest was strongest in CV	RF achieved the highest CV macro F1	Strong average stability across folds	Slightly below kNN on final test F1
Performance improved with size	Main and extended model result tables	More data improved class boundary estimation	Gains diminished near ceiling performance
MLP was scaling-sensitive	MLP scaler comparison table/figure	Scaling affected optimization and final performance	Differences were small overall
Shared features improved fairness	Controlled feature set across classes	Prevented class-specific feature advantage	Excluded potentially useful non-shared features
SVMs were less practical at scale	Extended model results/runtime note	Nonlinear kernels had higher runtime cost	Not all SVM variants run at 37,800 samples

10 Ethical, Legal, and professional Considerations

This project uses publicly available astronomical catalogue data and does not involve personal data, human participants, or sensitive demographic information. As a result, the direct ethical risk is low compared to many applied machine-learning projects.

The main ethical and legal considerations relate to correct data attribution, reproducibility, and reasonable reporting of model performance such as Gaia and the NASA Exoplanet Archive should be properly cited, and the results should not be presented as operationally validated beyond the datasets used in this dissertation.

A further consideration is scientific reliability. Very high classification scores can be misleading if caused by data leakage, duplicate sources, or catalogue-specific artifacts. For this reason, the project used features, stratified evaluation, and held-out testing, and future work should include external validation on independent catalogues.

Overall, the project presents low direct ethical risk, but reasonable scientific use requires transparent reporting of data sources, limitations, and validation boundaries.

11 Limitations and Future Work

Although the results show very strong multiclass performance, several limitations should be considered when interpreting the findings. The first limitation is the restricted shared-feature set. The experiment intentionally used only features available across all three classes: `ra`, `dec`, `parallax`, `pmra`, `pmdec`, and `phot_g_mean_mag`. This made the comparison fair, but it also excluded potentially informative class-specific features. As a result, the experiment evaluates model performance under controlled shared-feature conditions rather than the maximum possible performance using all available astronomical information.

Additional photometric colours, quality flags, uncertainty estimates, or derived astrophysical features could improve performance and provide more scientifically interpretable decision boundaries. However, richer feature engineering would need to be handled carefully to avoid leakage or class-specific bias. Future work could therefore test whether larger feature sets improve performance while still preserving a fair shared-feature design.

A second limitation is the three-class scope. The multiclass task was limited to `star`, `exo_host`, and `quasar`. This was suitable for building and evaluating a robust pipeline, but real astronomical catalogues contain many more object types. Adding classes such as galaxies, white dwarfs, variable stars, asteroids, and comets would make the classification task closer to real catalogue workflows and would test whether the current preprocessing and modelling pipeline scales beyond the controlled three-class setting.

A further limitation is the inclusion of sky-position features, `ra` and `dec`. These are valid astronomical catalogue fields, but they may also encode survey-selection effects or dataset-origin patterns if different classes are drawn from different sky regions. This means that part of the model performance could reflect catalogue sampling structure rather than only intrinsic object properties. Future work should repeat the multiclass experiment with `ra` and `dec` removed, or compare results with and without positional features, to test how much the classifiers depend on sky-location information.

Another limitation is the limited hyperparameter tuning. The models were configured using reasonable and consistent settings, but a full hyperparameter search was not the main focus of the project. This was appropriate for maintaining a fair and manageable comparison, but it means that some models may not have reached their best possible performance. Future work could use grid search, randomized search, or Bayesian optimization within the same cross-validation framework, especially for MLP, SVM, Random Forest, and kNN.

There is also a potential source-domain mismatch between datasets. Stars, quasars, and exoplanet hosts were collected and cleaned from different catalogue sources, and differences in source selection, measurement quality, or catalogue construction may influence model behaviour. Although schema alignment and shared-feature selection reduced this risk, the possibility remains that the model partly learn source-specific-patterns rather than only astrophysical differences.

Finally, the project did not use a separate external validation dataset. The held-out test split gives a useful estimate of generalisation within the assembled dataset, but it does not fully demonstrate performance on a completely independent survey or future catalogue release. A stronger validation design would test the final pipeline on an independent catalogue or later data release. This would help determine whether the model generalizes beyond the specific data preparation

choices used in this dissertation.

Future work should extend the project in several directions. First, additional object classes should be added to increase realism and difficulty. Second, richer feature engineering should be explored, including colour indices, uncertainty measures, and additional astrometric or photometric quality indicators. Third, gradient-boosting models such as XGBoost or LightGBM could be tested because they are often strong on structured tabular data. Fourth, deeper neural approaches using Keras or PyTorch could be investigated, especially if larger feature sets or image-based data are added. Finally, external validation should be performed using independent data to test robustness outside the current cleaned dataset.

Table 16: Limitations and proposed future improvements.

Limitation	**Impact**	**Proposed future improvement**
Shared-feature restriction	Ensures fairness but may exclude informative class-specific features	Add richer feature engineering while keeping comparisons controlled
Three-class scope	Simpler than real astronomical catalogue classification	Extend to more object classes such as asteroids, comets, galaxies, and other sources
Limited hyperparameter tuning	Some models may not be fully optimized	Use systematic grid search, random search, or Bayesian optimization
Dataset/source mismatch	Models may learn catalogue-specific patterns	Validate on independent catalogues or newer survey releases
No external validation set	Test split measures internal generalisation only	Evaluate final models on fully independent external data
SVM runtime constraints	Nonlinear SVM variants were not run at all dataset sizes	Use approximate kernels or more scalable classifiers

12 Conclusion

This dissertation investigated automated celestial-object classification using supervised machine learning on tabular astronomical data. The main aim was to build a fair and reproducible multiclass classification pipeline for three labels: `star`, `exo_host`, and `quasar`. The project focused not only on model performance, but also on data cleaning, schema alignment, feature selection, balanced sampling, and robust evaluation.

The final multiclass experiments used a shared Gaia-like feature set consisting of `ra`, `dec`, `parallax`, `pmra`, `pmdec`, and `phot_g_mean_mag`. Balanced subsets were evaluated at increasing sizes, and the final protocol used stratified 5-fold cross-validation on the train+validation partition plus an untouched held-out test set. This design helped reduce evaluation bias and supported fair comparison across model families.

The results showed that classical machine learning models performed extremely well on the selected structured dataset. kNN with MinMax scaling achieved the best held-out test macro F1 on the largest balanced subset, while Random Forest achieved the strongest cross-validation macro F1. Logistic Regression provided a stable but lower-performing linear baseline, and MLPClassifier improved with dataset size but did not exceed the strongest classical models. The confusion matrix for the best held-out model showed only two errors, both involving `exo_host` samples predicted as `star`.

The research questions can therefore be answered directly. First, kNN with MinMax scaling achieved the best held-out test macro F1, while Random Forest achieved the strongest cross-validation macro F1. Second, performance generally improved as dataset size increased, although gains became smaller at larger sizes because the strongest models were already close to ceiling performance. Third, scaling choice had a measurable but not dominant effect on MLP performance, with StandardScaler stronger at smaller sizes and MinMaxScaler stronger on the largest held-out test result.

The project also shows that careful preprocessing is as important as model selection. The high final performance was not only the result of choosing a strong classifier; it depended on cleaning the datasets, aligning columns, controlling missing values, balancing the classes, preventing leakage, and evaluating with

stratified cross-validation and a held-out test set. These steps made the reported comparison more reliable and easier to defend.

Overall, the project demonstrates that careful preprocessing and fair experimental design are critical for reliable astronomical classification. The strong results suggest that tabular Gaia-like features can provide substantial class-discriminative information for this three-class problem. Future work should extend the pipeline to more object classes, richer features, stronger tabular models such as gradient boosting, and external validation on independent catalogues.

References

Akeson, R. L., X. Chen, D. Ciardi, *et al.* (2013). "The NASA Exoplanet Archive: Data and Tools for Exoplanet Research". In: *Publications of the Astronomical Society of the Pacific* 125.930, p. 989. DOI: 10.1086/672273.

Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Springer. DOI: 10.1007/978-0-387-45528-0.

Breiman, Leo (2001). "Random Forests". In: *Machine Learning* 45.1, pp. 5–32. DOI: 10.1023/A:1010933404324.

Cover, T. M. and P. E. Hart (1967). "Nearest Neighbor Pattern Classification". In: *IEEE Transactions on Information Theory* 13.1, pp. 21–27. DOI: 10.1109/TIT.1967.1053964.

ESA Gaia Archive (2023). *Gaia DR3 documentation: gaia_source data model*. https://gea.esac.esa.int/archive/documentation/GDR3/Gaia_archive/chap_datamodel/sec_dm_main_source_catalogue/ssec_dm_gaia_source.html. Accessed: 2026-04-22.

Gaia Collaboration (2016). "The Gaia mission". In: *Astronomy & Astrophysics* 595, A1. DOI: 10.1051/0004-6361/201629272.

Gaia Collaboration *et al.* (2023). "Gaia Data Release 3: Summary of the content and survey properties". In: *Astronomy & Astrophysics* 674, A1. DOI: 10.1051/0004-6361/202243940.

James, Gareth *et al.* (2021). *An Introduction to Statistical Learning: with Applications in R*. 2nd ed. Springer. DOI: 10.1007/978-1-0716-1418-1.

Kohavi, Ron (1995). "A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection". In: *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*. Vol. 2. Morgan Kaufmann, pp. 1137–1143. DOI: 10.5555/1643031.1643047.

NASA Exoplanet Archive (2026). *Planetary Systems and Planetary Systems Composite Parameters: Data Column Definitions*. https://exoplanetarchive.ipac.caltech.edu/docs/API_PS_columns.html. Accessed: 2026-04-22.

Pedregosa, Fabian *et al.* (2011). "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12, pp. 2825–2830. URL: https://www.jmlr.org/papers/v12/pedregosa11a.html.

scikit-learn developers (2026a). *confusion_matrix documentation*. https://sklearn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html. Accessed: 2026-04-22.

— (2026b). *Cross-validation: evaluating estimator performance*. https://sklearn.org/stable/modules/cross_validation.html. Accessed: 2026-04-22.

— (2026c). *MinMaxScaler documentation*. https://sklearn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html. Accessed: 2026-04-22.

— (2026d). *MLPClassifier documentation*. https://sklearn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html. Accessed: 2026-04-22.

— (2026e). *precision_recall_fscore_support documentation*. https://sklearn.org/stable/modules/generated/sklearn.metrics.precision_recall_fscore_support.html. Accessed: 2026-04-22.

— (2026f). *StandardScaler documentation*. https://sklearn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html. Accessed: 2026-04-22.

— (2026g). *StratifiedKFold documentation*. https://sklearn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html. Accessed: 2026-04-22.

— (2026h). *train_test_split documentation*. https://scikit-learn.org/dev/modules/generated/sklearn.model_selection.train_test_split.html. Accessed: 2026-04-22.

Sokolova, Marina and Guy Lapalme (2009). "A Systematic Analysis of Performance Measures for Classification Tasks". In: *Information Processing & Management* 45.4, pp. 427–437. DOI: 10.1016/j.ipm.2009.03.002.

A Additional Multiclass Results

This appendix contains additional multiclass plots that support the main Results chapter. The main report includes the key summary plots and confusion matrix; the figures below provide supplementary precision, recall, and model-comparison views.

A.1 All-Model Metric Comparisons

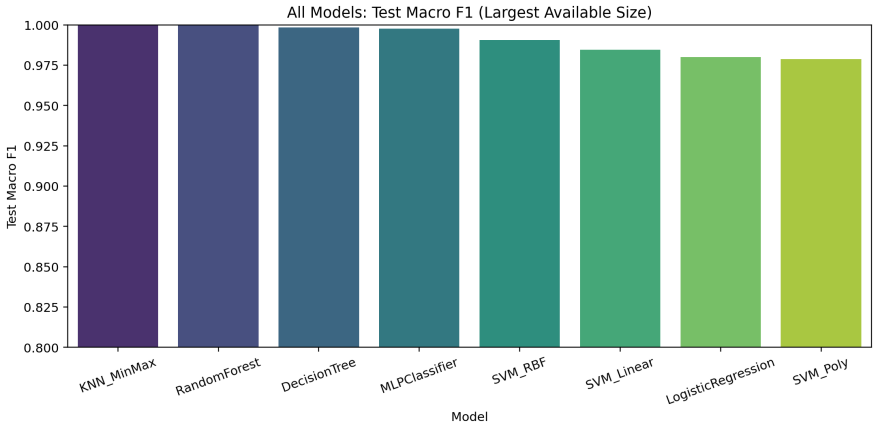

Figure 10: Held-out test macro F1 for each model using the largest available evaluated subset.

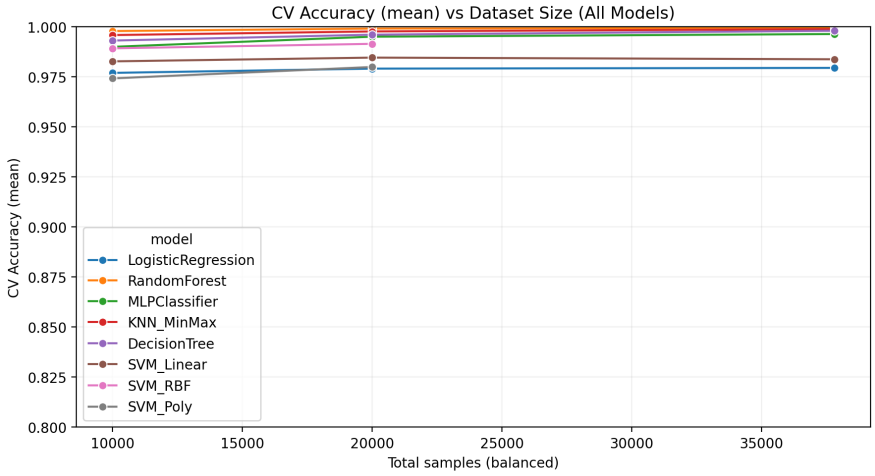

Figure 11: Cross-validation accuracy versus dataset size for all evaluated multiclass models.

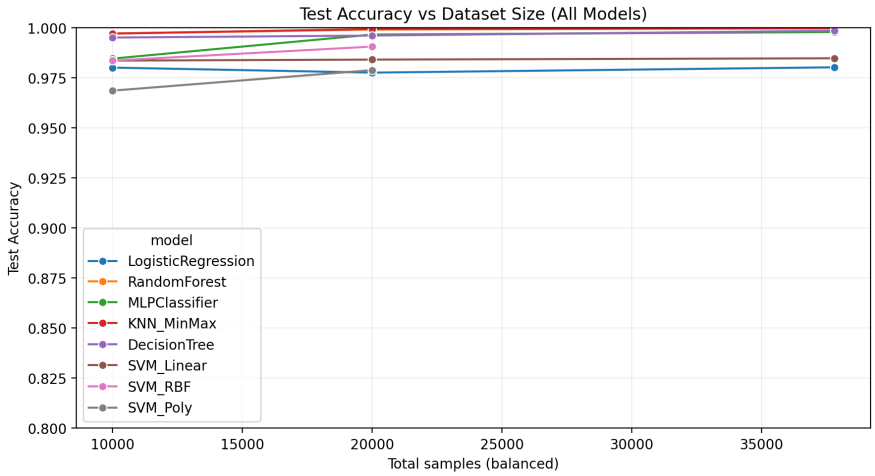

Figure 12: Held-out test accuracy versus dataset size for all evaluated multiclass models.

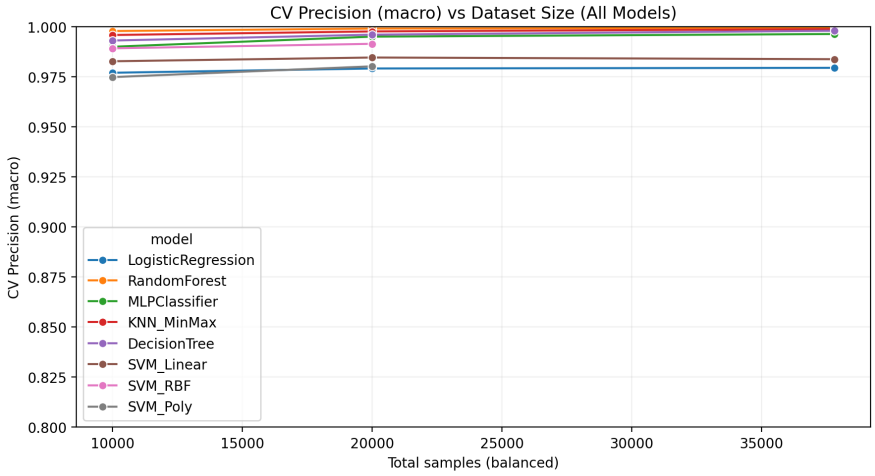

Figure 13: Cross-validation macro precision versus dataset size for all evaluated multiclass models.

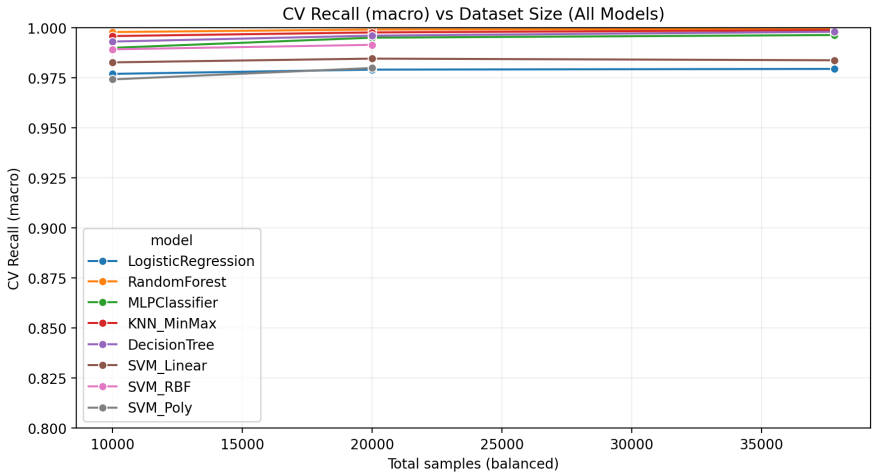

Figure 14: Cross-validation macro recall versus dataset size for all evaluated multiclass models.

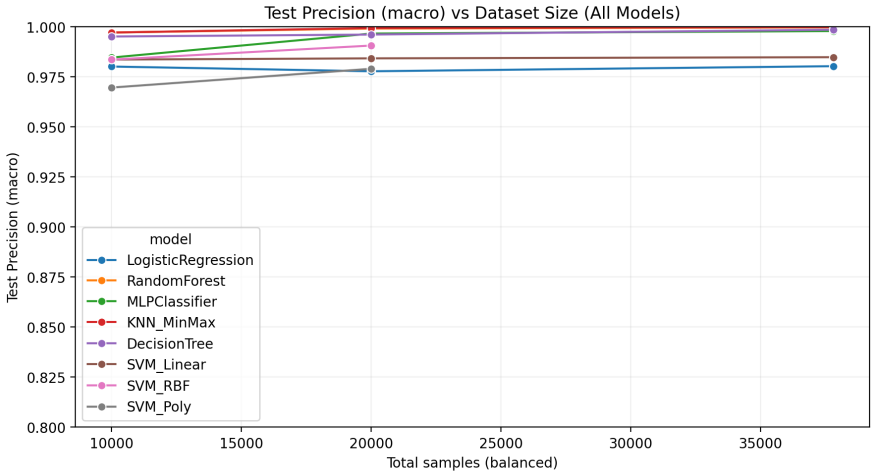

Figure 15: Held-out test macro precision versus dataset size for all evaluated multiclass models.

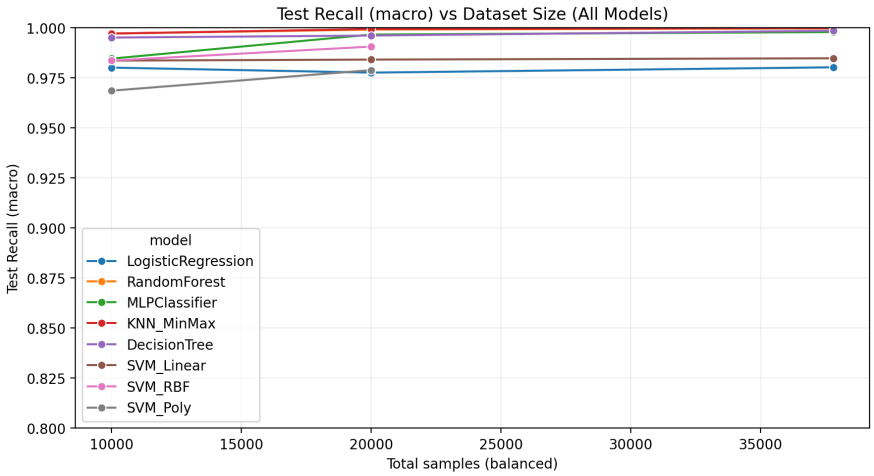

Figure 16: Held-out test macro recall versus dataset size for all evaluated multiclass models.

A.2 MLP Scaler Comparison

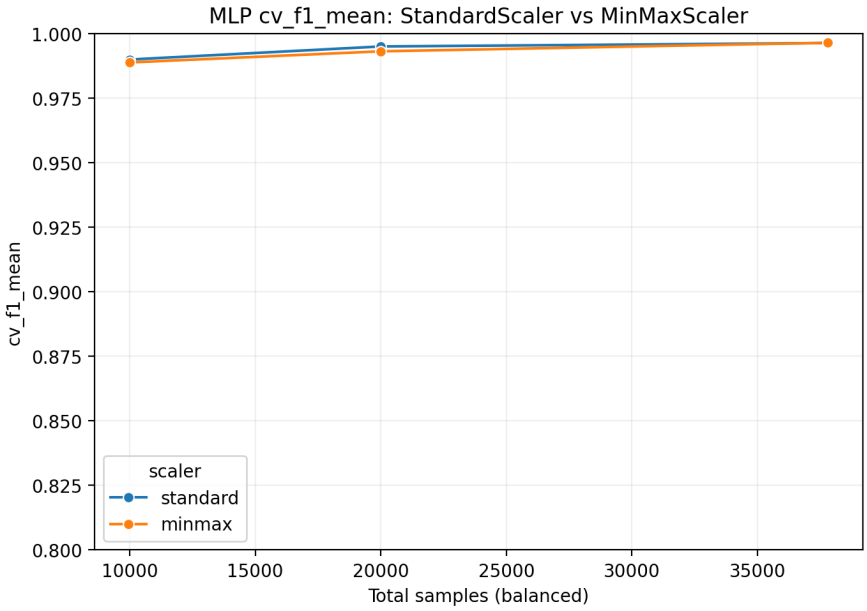

Figure 17: MLP cross-validation macro F1 comparison between StandardScaler and MinMaxScaler.

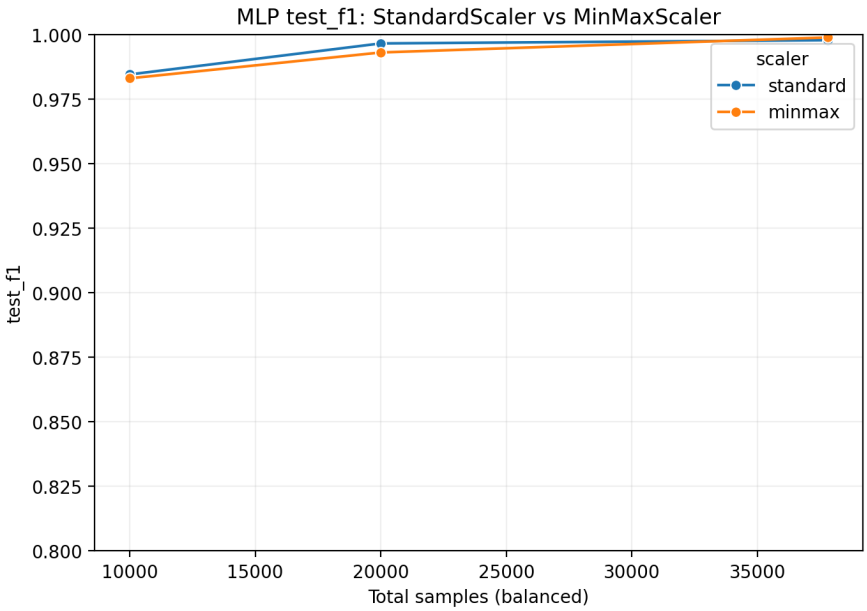

Figure 18: MLP held-out test macro F1 comparison between StandardScaler and MinMaxScaler.

A.3 Additional Confusion Matrices

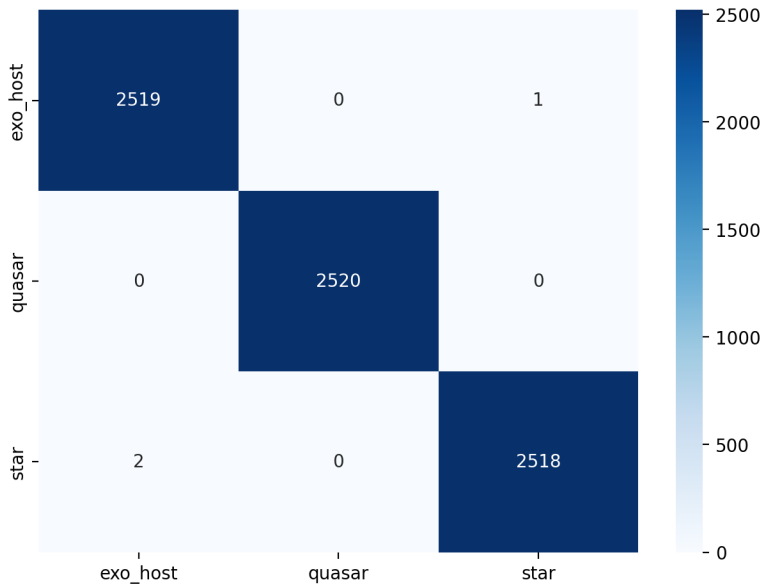

Figure 19: Random Forest confusion matrix on the largest balanced multiclass subset.

B Preliminary Binary Experiment Outputs

The binary experiments were conducted earlier in the project using smaller legacy or exploratory datasets. They are included here as supplementary pipeline-development outputs and are not directly compared with the final multiclass experiments.

B.1 Asteroids vs Comets

B.2 Host Stars vs Non-host Stars

B.3 Quasars vs Stars

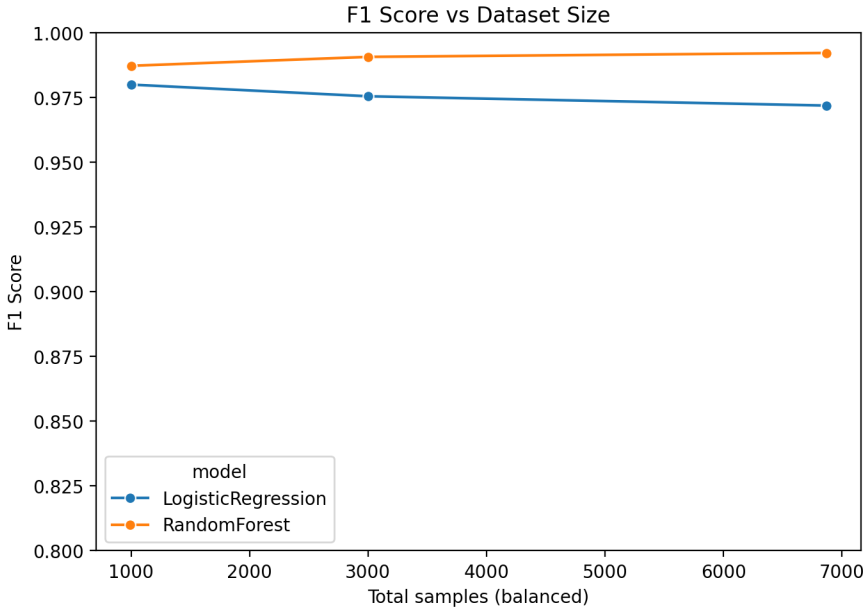

Figure 20: Supplementary result plot for the preliminary asteroid-versus-comet binary experiment.

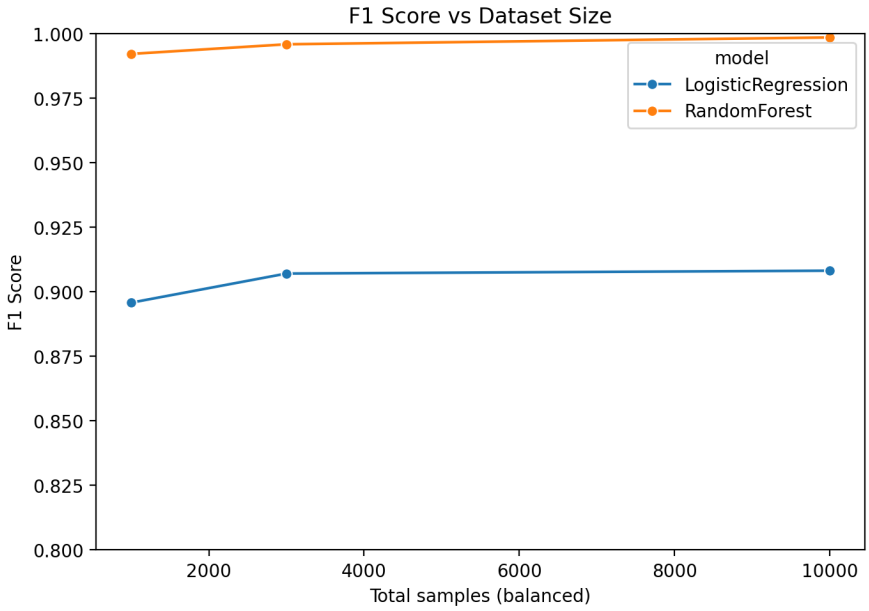

Figure 21: Supplementary result plot for the preliminary host-star versus non-host-star binary experiment.

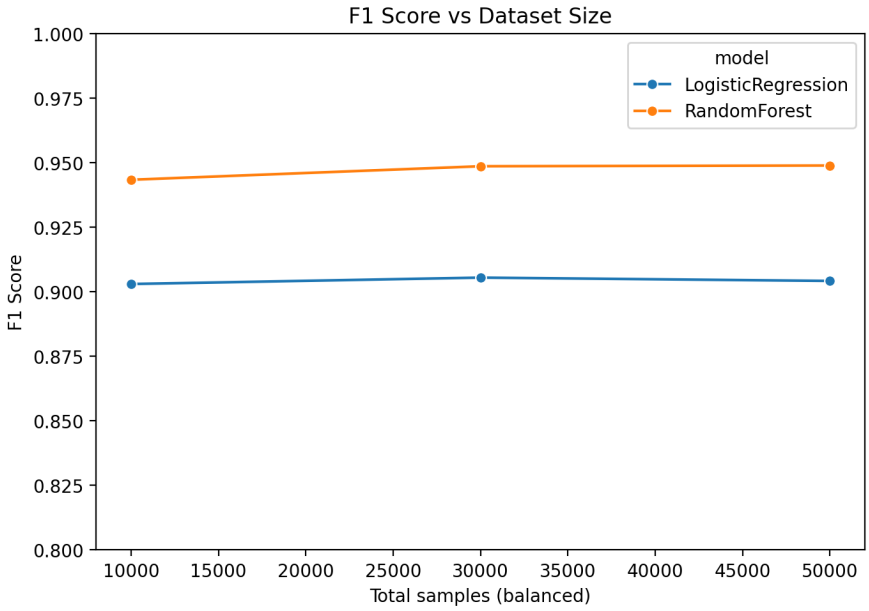

Figure 22: Supplementary result plot for the preliminary quasar-versus-star binary experiment.

C Generative AI Declaration

Student Declaration of AI Tool use in this Assessment

Please indicate your level of usage of generative AI for this assessment - please tick the appropriate category(s).

If the “Assisted Work” or “Partnered Work” category is selected, please expand on the usage and in which elements of the assignment the usage refers to.

Solo Work	**S1 - Generative AI tools have not been used for this assessment.**	☐
Assisted Work	**A1 – Idea Generation and Problem Exploration** Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.	☒
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student’s own work.	☒
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student’s own work.	☐
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions. The interpretation and integration of research into the assignment remain the student’s responsibility.	☐
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.	☒
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.	☐
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.	☐
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.	☒

	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.	☒
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.	☐
	A11 - Other uses not listed above Please specify:	☐
Partnered Work	**P1 - Generative AI tool usage has been used integrally for this assessment** Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:	☐

Please provide details of AI usage and which elements of the coursework this relates to: Generative AI tools were used in an assistive role during the project. The support related to report structuring, language refinement, grammar checking, technical guidance, debugging support, and validation support. Specifically, AI was used to help plan dissertation sections, improve clarity of written explanations, check grammar and spelling, discuss machine-learning methodology, interpret coding/debugging issues, and review whether the experimental design and results were presented clearly.

AI was not used as a replacement for the project work. The dataset preparation, notebook execution, model training, result generation, interpretation of final results, and final submitted content were reviewed, edited, and verified by me. All decisions about the project design, implementation, analysis, and final report remained my responsibility.

I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.	☒
I confirm that all details provide above are an accurate description of how AI was used for this assessment.	☒